SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering

A.Y. 2025 - 26

Course: Database Management Systems

Project Report

Program	Btech CSEDS-311	
Semester	4	
Name of the Project:	SQL Analytics Dashboard	
Details of Project Members		
Batch: B1	Roll No. L030	Name: Krissh Khaneja
Batch: B1	Roll No. L005	Name: Ruturaj Bannati
Batch: B1	Roll No. L019	Name: Ritwik Goyal
Date of Submission: 20/4/26		

Contribution of each project Members:

Roll No.	Name:	Contribution
L030	Krissh Khaneja	Base SQL functions implementation(Insert/update/join etc), Set operations functions
L005	Ruturaj Bannati	Frontend using streamlit, normalization function
L019	Ritwik Goyal	ER Diagram mapping, transaction function

Github link of your project:

https://github.com/krisshk123/DBMS-

Project Report

Selected Topic

by

Krissh Khaneja, Roll number: L030

Ruturaj Bannati, Roll number: L005

Ritwik Goyal, Roll number: L019

Course: DBMS

AY: 2025-26

Table of Contents

Sr no.	Topic	Page no.
1	Problem statement	
2	Components of Database Design	
3	Entity Relationship Diagram	
4	Relational Model	
5	Normalization	
6	SQL Queries	
7	Learning from the Project	
8	Project Demonstration	
9	Self-learning beyond classroom	
10	Learning from the project	
8	Challenges faced	
9	Conclusion	

I. Problem Statement

This project aims at designing and developing a well-rounded and interactive database management system that illustrates the implementation of all the fundamental concepts learned in the Database Management Systems course. Instead of creating a project within a domain problem, the system is designed as a complete-fleet analytics dashboard that uses all the key areas of the curriculum - database design, SQL querying, data manipulation, transaction control, set theory, relational algebra, functional dependencies and normalization - in a single application.

The data that will be used in this project is the Olist Brazilian e-commerce dataset. It offers a realistic relational data in the form of multi-table with natural relationships among the entities of customers, orders, products, sellers, payments, and reviews. This is why it is quite appropriate to show joins among several tables, aggregation queries, referential integrity, and normalization analysis, which cannot be meaningful without a rich enough schema.

It is written in Python, SQLite, and Streamlit, and offers a graphical interface that runs in a browser, where all the course concepts can be interactively shown. The CSV files are loaded into an in-memory SQLite database during the run and all operations are immediately done on this database. The next course topics are each modeled as a specific module or operation in the system.

The application is based on SQL querying. End-users can show records in any loaded table, use column-based filters, and run GROUP BY operations with aggregate functions (SUM, AVG, COUNT, MAX, MIN), and join any two tables based on a specified pair of columns. The combination of these operations is a description of the DDL level of schema awareness and the entire spectrum of DML read operations that are covered in class.

Data Manipulation Language is illustrated by special INSERT, UPDATE and DELETE operations, which are presented by means of a form-based interface. One can add new records to any table, update the values of existing records given a condition or delete rows that satisfy a given predicate - directly using the DML syntax and semantics learned in the course.

The implementation of Transaction Control Language is as a stand-alone module that takes the user through the entire transaction lifecycle. Users may BEGIN a transaction, create one or more DMLs which remain pending, create named SAVEPOINTs at intermediate points, ROLLBACK to a particular savepoint or roll back all pending changes and ultimately COMMIT to permanently store the results. Every step is recorded in a live transaction log, and the atomicity and isolation properties of transactions are observed in real time.

Set operations are illustrated by enabling the user to choose column sets of any two loaded tables and calculate their union, intersection, difference (A-B and B-A) and symmetric difference. The results are presented as a set of column names with a live preview of data when used, basing the abstract set-theoretic operations that are investigated in relational algebra on a practical, data-driven model.

The concept of functional dependencies and normalization is supported by a special module called Normalization Calculator. The user chooses a relation schema of any loaded table, manually enters a set of functional dependencies, and the system calculates all the candidate key by attribute closure, determines the primary key, classifies prime and non-prime attributes, detects partial dependency violations (2NF) or transitive

dependency violations (3NF), performs a lossless decomposition into the target normal form, and determines whether all original functional dependencies are maintained across the decomposed relations.

These modules are a full execution of the DBMS curriculum on a real-world dataset, an end-to-end implementation of a working software system.

II. Components of Database Design

A. Entities and Attributes.

1. Customer Refers to a buyer who is registered on the Olist platform. The customer unique id feature is used to differentiate between the physical person and the transactional record and one physical person can make numerous orders without recording identity information.

customer_id (Primary Key) — a unique identifier of a customer record which is a foreign key in the Order table.

customer unique id- a consistent cross order identifier of the same physical customer.

customer.zip.code.prefix — five-digit zip code prefix of the registered address of the customer.

customer_city - the city of the customer address where he is registered.

customer state - Two letter state code of the location of the customer.

2. Geolocation This is the data of geographic coordinates, which is projected onto zip code prefixes. This table can be used to perform spatial analysis of the distribution of customers and sellers in Brazil and it can be used as a reference when performing location based queries on the Customer and Seller entities.

geolocation_zip_code_prefix (Primary Key) - the five-digit zip code prefix that is used as the geographic anchor.

geolocation_lat Latitude of the area of this zip code.

geolocation_lng — longitude of this area of zip code.

geolocation_city — city name of this zip code prefix

geolocation_state - two-letter state abbreviation of this zip code prefix.

3. Order Represents a customer purchase transaction. Order is the core of the schema, which connects customers to items, payments, and reviews. It documents the entire lifecycle of a transaction by a status field and a sequence of timestamps representing each step of fulfillment.

order_id (Primary Key) - primary key of the order.

customer_id (Foreign Key) - refers to the Customer who made the order.

order status - current position of the order: created, approved, invoiced, processing, shipped, delivered, unavailable, or canceled

order_purchase_timestamp - date and time when the customer made the order.

order approved at - date and time payment was approved.

order-delivered-carrier-date 1 - date and time when the order was given to the logistics carrier.

order_delivered_customer date- actual date and time when the customer received the order.

order_estimated_delivery_date — delivery date indicated to customer during purchase.

4. Order Item is a single line of a product in an order. One order can include several items that are provided by different sellers and refer to different products, and they have their own price, freight cost, and shipping deadline.

order_id (Foreign Key, composite PK) - order which this item is part of.

order item id (part of composite PK) - sequential integer that gives the position of the item in the order.

product_id (Foreign Key) - the product that is being bought.

seller_id (Foreign Key) - the seller of this item.

shipping_limit_date — date before which this item has to be delivered by the seller to the carrier.

price - unit price of product of this line item

freight_value - shipping price of this item.

Primary Key: (order_id, order_item_id) - composite, as item numbers re-begin with 1 per order.

5. Order Payment Represents a payment record of an order. An order can be paid in various ways or can be divided into installments and will result in more than one row of payment on an order. A row records one payment part including its method, installment number and value.

order id (Foreign Key, composite PK) - the order that this payment is linked to.

payment_sequential (composite PK) - sequential value representing the order of this payment in the order.

payment-type — the type of payment: credit-card, boleto, voucher or debit-card.

payment_installments — number of installments that the customer has selected on this segment.

payment_value - financial value of this portion of payment.

Primary Key: (order_id, payment_sequential) - composite, as payment_sequential only makes sense in a particular order.

6. Order Review Represents a post delivery review made by a customer on an already processed order. Customers will use a scale of 1-5 to rate their experience and can choose to leave a written comment. The reviews are time stamped during its creation and submission.

review_id (Primary Key) — identifier of the review.

order_id (Foreign Key) -the order under examination.

review score - customer satisfaction rating 1-5 (lowest to highest)

review_comment_title - short title given by the customer (optional).

review_comment_message — the text of the written review given by the customer (not all customers provide text).

review creation date - date the review request was dispatched to the customer.

review_answer_timestamp - the date and time when the review was sent.

7. Product Represents a product in Olist catalog. The Portuguese classifies products and defines them using a list of dimensional and content attributes which are useful in logistics, freight calculation, and listing quality analysis.

product_id (Primary Key) - unique identifier of the product.

product_category_name - category of the item in Portuguese, can be added to the translation table.

product_name lengtheness character length of the product name

product_description_lenght - character length of a complete description of the product.

product_photos_qty — quantity of photos on the product listing.

product weight in grams is product weight weight in g.

product length cm — length of the product in cm.

product_height_cm product height in centimeters.

product width cm - width of product in centimeters.

8. Seller Represents a business signed to the platform that fulfills order items. The geographic profile of sellers is independent of customers and allows studying the concentration of sellers by area and regional logistics processes.

seller_id (Primary Key) — unique number of the seller.

seller_ zip code prefix- five digits zip code prefix of the business location of the seller.

seller_city — city in which the seller is located.

seller state- Abbreviation of the location of the seller in two letters.

9. Product Category Name Translation The product category names in Portuguese are translated into English in a lookup table. This table is used to have the product category data available to be reported in a wider manner without changing the actual product records.

product category name (Primary Key) - the Portuguese category name, which corresponds to the values in the Product table.

product_category_name_english — the English name of the category (e.g., health_beauty, computers_accessories)

B. Relationships Among Entities

1. Customer places Order Customer can make many orders throughout the time, although each Order is related to only one Customer. The order lifecycle begins with this relationship, which allows analytics on a customer level, including purchase frequency and geographic location of customers.

Cardinality: One-to-Many (1:N) - a Customer can make a number of Orders; an Order can only be made by one Customer.

Participation: Partial Customer (a registered customer might not have made any orders); Total Order (any order has to be linked with a customer).

2. Order includes Order Item An Order consists of a sequence of one or more Order Items, each representing a unique combination of product and seller that is being bought in that order. This relationship reflects the line-item format of a shopping cart, and is the focus of revenue, freight and product-level analysis.

Cardinality: One-to-Many (1:N) — there is one Order and many Order Items; each Order Item is a part of a single Order.

Participation: Complete on both sides - it takes at least one item in an Order to be meaningful and all Order Items must be part of an Order.

3. Order has Order Payment An Order is paid by a single or a series of Payment records, which supports split payment schemes, and instalment schemes. This relationship can be fully reconstructed to determine how a customer decided to pay an order on a part of any order.

Cardinality: One-to-Many (1:N) — an Order can have an Order.Purchase record, or a Payment record; a record of a purchase can only be in an Order.

Participation: Complete on both sides - all orders should have one or more payment records; all payment records should have a valid order referenced.

4. Order receives Order Review Customer may give a Review of their Order after delivery. Order-Review: there is a 1-1 relationship between Reviews and Orders.

Cardinality: One-to-One (1:1) — there is no more than one Order per one Review; there exists one Order per one Review.

Participation: Part on the Order side (not all orders delivered are reviewed); Complete on the Review side (all reviews mention the order being analyzed).

5. Order Item refers to a Product Each Order Item refers to a particular Product in the catalog. One Product can be represented in numerous Order Items in numerous different orders over time, and this is where this relationship plays a crucial role in the analysis of product popularity and sales volume.

Cardinality: Many-to-One (N:1) - there can be many Order Items that refer to the same Product; one Order Item can refer to only one Product.

Participation: Partial on Product (a product can be listed, but never purchased); Total on Order Item (each order item must mention a valid product).

6. Order Item is fulfilled by Seller Each Order Item is fulfilled by a particular Seller. A Seller can complete numerous Order Items on numerous orders and at most, each Order Item has a single Seller who fulfills it.

Cardinality: Many-to-One (N:1) there are many Order Items that can be fulfilled by the same Seller; every Order Item is filled by only one Seller.

Participation: Partial on the Seller side (a registered seller is not allowed to have any active order items); Total on the Order Item side (each order item is required to have an assigned seller).

7. Customer and Seller are in the Geolocation Both Customer and Seller entities are connected to the Geolocation table by their zip code prefix attributes. This relationship allows geographic mapping and analysis of delivery by distance. One zip code prefix record in Geolocation can have a number of customers and numerous sellers.

Cardinality: Many-to-One (N:1) of Customer→Geolocation and Seller→Geolocation - a single zip code prefix entry can be shared by many customers or sellers.

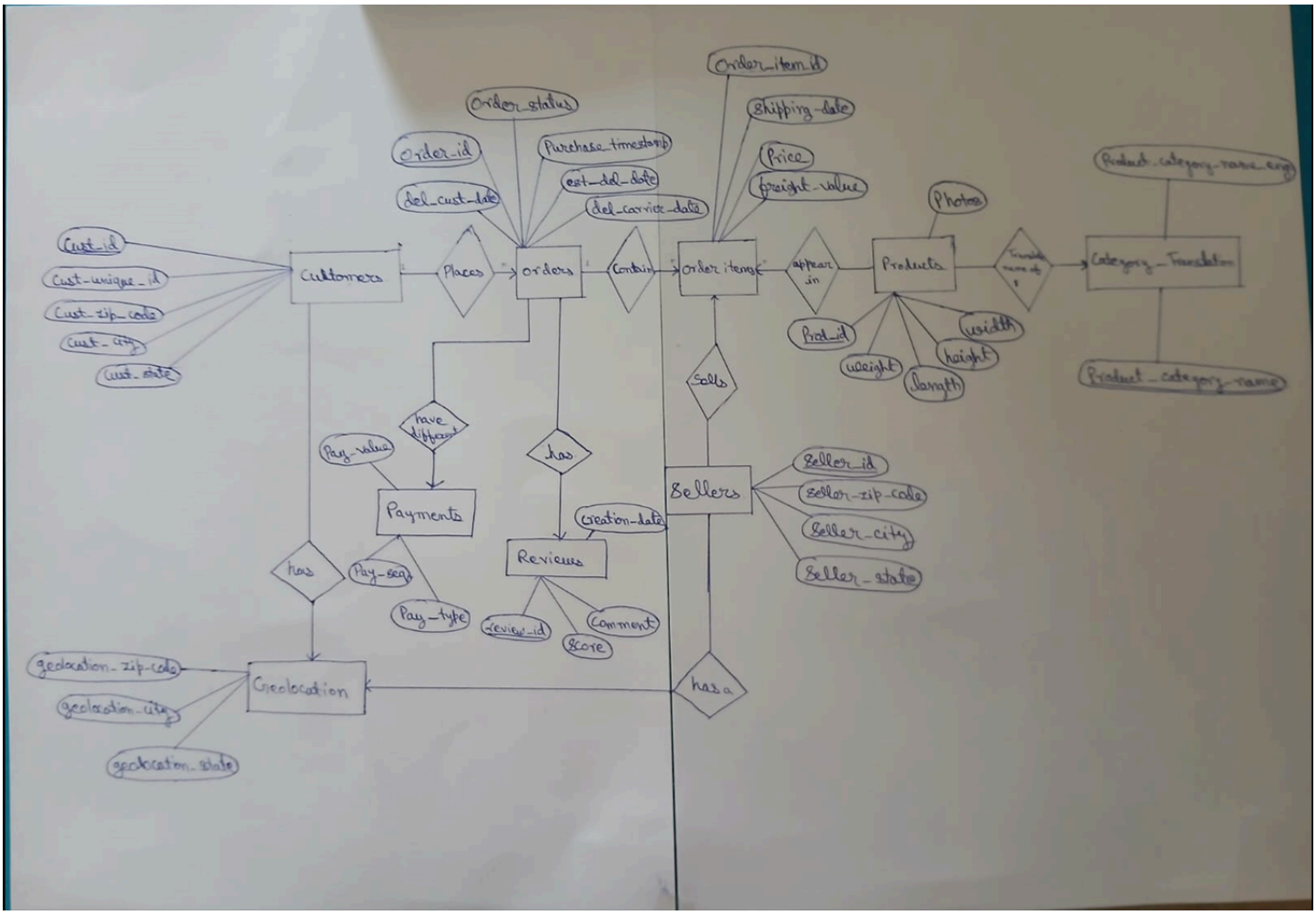
Participation: Partial on the Geolocation side (not all zip codes prefixes recorded in the table have a registered customer or seller); Partial on both Customer and Seller side (zip code data can be unavailable or unmatched on the geolocation table).

8. Product is a Product Category Name Translation A Product is a category name in Portuguese, which is translated into an English equivalent by the translation table. This association allows reporting on the category level in English without changing the initial product records.

Cardinality: many-to-one (N:1) - a large number of Products can be of the same category, but a Product can refer to only one category name.

Participation: Partial on the side of translation table (not all the categories of the translation table have products associated with it); Partial on the Product side (not all products have a null or unnamed category name).

III. Entity Relationship Diagram



IV. Relational Model

CUSTOMERS

(customer_id PK, customer_unique_id, customer_zip_code_prefix FK -> GEOLOCATION, customer_city, customer_state)

GEOLOCATION

(geolocation_zip_code_prefix PK, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state)

SELLERS

(seller_id PK, seller_zip_code_prefix FK -> GEOLOCATION, seller_city, seller_state)

PRODUCTS

(product_id,PK,product_category_nameFK>PRODUCT_CATEGORY_NAME_TRANSLATION, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm)

PRODUCT_CATEGORY_NAME_TRANSLATION

(product_category_name PK, product_category_name_english)

ORDERS

(order_id PK, customer_id FK -> CUSTOMERS, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date)

ORDER_ITEMS

(order_id FK -> ORDERS, order_item_id, product_id FK -> PRODUCTS, seller_id FK -> SELLERS, shipping_limit_date, price, freight_value) PK: (order_id, order_item_id)

ORDER_PAYMENTS

(order_id FK -> ORDERS, payment_sequential, payment_type, payment_installments, payment_value) PK: (order_id, payment_sequential)

ORDER_REVIEWS

(review_id PK, order_id FK -> ORDERS, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp)

V. Normalization

Normalization is applied to each of the nine relations in the Olist database. For each table, the analysis covers 1NF, 2NF, 3NF, and BCNF in sequence, identifying violations where they exist and stating the final normalized form achieved.

1. Customer

Relation: Customer(customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state)

Functional Dependencies:

- customer_id → customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
- customer_zip_code_prefix → customer_city, customer_state

Candidate Keys: {customer_id} **Primary Key:** customer_id

1NF: All attributes are atomic and single-valued. There are no repeating groups or multi-valued columns. The table is in 1NF.

2NF: The primary key is a single attribute (customer_id), so partial dependencies are impossible by definition. The table is in 2NF.

3NF: A transitive dependency exists: customer_id → customer_zip_code_prefix, and customer_zip_code_prefix → customer_city, customer_state. This means customer_city and customer_state are transitively dependent on the primary key through zip code, not directly. This violates 3NF.

Decomposition into 3NF:

Customer(customer_id, customer_unique_id, customer_zip_code_prefix)
CustomerZip(customer_zip_code_prefix, customer_city, customer_state)

BCNF: After decomposition, in the Customer relation the only functional dependency is customer_id → customer_unique_id, customer_zip_code_prefix, where customer_id is a superkey. In CustomerZip, customer_zip_code_prefix → customer_city, customer_state, where customer_zip_code_prefix is a superkey. Both relations satisfy BCNF. Note that CustomerZip overlaps with the Geolocation table already present in the schema, so in practice this decomposition consolidates into the existing Geolocation relation rather than creating a redundant table.

Final Form: BCNF

2. Geolocation

Relation: Geolocation(geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state)

Functional Dependencies:

- geolocation_zip_code_prefix → geolocation_city, geolocation_state
- (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng) → geolocation_city, geolocation_state

Candidate Keys: {geolocation_zip_code_prefix} (treating each zip prefix as identifying a unique city-state pair) **Primary Key:** geolocation_zip_code_prefix

1NF: All attributes are atomic. However, the dataset contains multiple rows for the same zip code prefix with different latitude and longitude values, representing different coordinate readings for the same zip area. This means the table has duplicate zip code prefix values, which technically violates the requirement for a unique primary key. To satisfy 1NF properly, a surrogate primary key or a composite key of (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng) must be used to uniquely identify each row.

Revised Primary Key: (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng)

The table is now in 1NF.

2NF: With the composite primary key (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng), geolocation_city and geolocation_state depend only on geolocation_zip_code_prefix alone — a proper subset of the composite key. This is a partial dependency and violates 2NF.

Decomposition into 2NF:

GeoCoordinates(geolocation_zip_code_prefix, geolocation_lat, geolocation_lng)
GeoLocation(geolocation_zip_code_prefix, geolocation_city, geolocation_state)

3NF: In GeoCoordinates, the only dependency is the composite key determining the record itself — no transitive dependencies exist. In GeoLocation, geolocation_zip_code_prefix → geolocation_city, geolocation_state with no further transitive chain. Both relations are in 3NF.

BCNF: In GeoCoordinates, the composite key is the only determinant. In GeoLocation, geolocation_zip_code_prefix is a superkey and the only determinant. Both satisfy BCNF.

Final Form: BCNF (after decomposition from 1NF violation)

3. Order

Relation: Order(order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date)

Functional Dependencies:

- order_id → customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date

Candidate Keys: {order_id} **Primary Key:** order_id

1NF: All attributes are atomic and single-valued. No repeating groups exist. The table is in 1NF.

2NF: The primary key is a single attribute, making partial dependencies impossible. The table is in 2NF.

3NF: All non-key attributes depend directly and solely on order_id. There are no non-key attributes that determine other non-key attributes. Although the timestamp columns are related in meaning (a logical sequence of events), none of them functionally determines another — each is an independently recorded value. The table is in 3NF.

BCNF: The only determinant in the relation is order_id, which is the primary key and therefore a superkey. No non-superkey determinants exist. The table is in BCNF.

Final Form: BCNF

4. Order Item

Relation: OrderItem(order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value)

Functional Dependencies:

- (order_id, order_item_id) → product_id, seller_id, shipping_limit_date, price, freight_value
- product_id → price (a given product has a listed price, though sellers may vary it)
- (product_id, seller_id) → price, freight_value (price and freight are determined by the product-seller combination)

Candidate Keys: {(order_id, order_item_id)} **Primary Key:** (order_id, order_item_id)

1NF: All attributes are atomic and single-valued. No repeating groups exist. The table is in 1NF.

2NF: All non-key attributes — product_id, seller_id, shipping_limit_date, price, freight_value — depend on the full composite key (order_id, order_item_id) and not on either component alone. order_item_id alone has no meaning without order_id, and order_id alone does not determine which product or seller is involved. There are no partial dependencies. The table is in 2NF.

3NF: The price and freight_value of an item are determined by the specific product-seller combination rather than solely by the order context. This introduces a transitive dependency: (order_id, order_item_id) → (product_id, seller_id) → price, freight_value. This violates 3NF.

Decomposition into 3NF:

OrderItem(order_id, order_item_id, product_id, seller_id, shipping_limit_date)
SellerProductPricing(product_id, seller_id, price, freight_value)

BCNF: In OrderItem, the only determinant is the composite primary key (order_id, order_item_id), which is a superkey. In SellerProductPricing, the determinant (product_id, seller_id) is the primary key and therefore a superkey. Both relations satisfy BCNF.

Final Form: BCNF (after decomposition from 3NF violation)

5. Order Payment

Relation: OrderPayment(order_id, payment_sequential, payment_type, payment_installments, payment_value)

Functional Dependencies:

- (order_id, payment_sequential) → payment_type, payment_installments, payment_value

Candidate Keys: {(order_id, payment_sequential)} **Primary Key:** (order_id, payment_sequential)

1NF: All attributes are atomic. payment_type holds a single value per row (credit_card, boleto, etc.), and there are no multi-valued or repeating fields. The table is in 1NF.

2NF: All non-key attributes depend on the full composite key. payment_type, payment_installments, and payment_value are meaningful only in the context of a specific payment entry within a specific order — neither order_id nor payment_sequential alone determines them. There are no partial dependencies. The table is in 2NF.

3NF: There are no non-key attributes that determine other non-key attributes. payment_installments and payment_value are independent facts about the payment entry and do not transitively determine one another. The table is in 3NF.

BCNF: The only determinant is (order_id, payment_sequential), which is the primary key and a superkey. No other determinants exist. The table is in BCNF.

Final Form: BCNF

6. Order Review

Relation: OrderReview(review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp)

Functional Dependencies:

- review_id → order_id, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp
- order_id → review_id (since each order has at most one review, order_id also uniquely identifies the review)

Candidate Keys: {review_id}, {order_id} **Primary Key:** review_id

1NF: All attributes are atomic and single-valued. review_comment_title and review_comment_message may be null but are still single-valued per row. No repeating groups exist. The table is in 1NF.

2NF: The primary key is a single attribute (review_id), so partial dependencies are impossible. The table is in 2NF.

3NF: All non-key attributes depend directly on review_id with no transitive chain. order_id is itself a candidate key, so its presence does not introduce a transitive dependency — it is simply an alternate key. The table is in 3NF.

BCNF: Both determinants in the relation — review_id and order_id — are candidate keys and therefore superkeys. Every non-trivial functional dependency has a superkey on its left-hand side. The table is in BCNF.

Final Form: BCNF

7. Product

Relation: Product(product_id, product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm)

Functional Dependencies:

- product_id → product_category_name, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm
- product_category_name → (no further attributes in this table; category name does not determine any physical dimension)

Candidate Keys: {product_id} **Primary Key:** product_id

1NF: All attributes are atomic. Physical dimensions, weights, and counts are each stored as individual scalar values with no repeating groups. The table is in 1NF.

2NF: The primary key is a single attribute, ruling out partial dependencies. The table is in 2NF.

3NF: All non-key attributes — category name, textual content lengths, photo count, and physical dimensions — depend solely and directly on product_id. product_category_name does not determine any of the physical attributes, so no transitive dependencies exist. The table is in 3NF.

BCNF: The only determinant is product_id, which is the primary key and a superkey. The table is in BCNF.

Final Form: BCNF

8. Seller

Relation: Seller(seller_id, seller_zip_code_prefix, seller_city, seller_state)

Functional Dependencies:

- seller_id → seller_zip_code_prefix, seller_city, seller_state
- seller_zip_code_prefix → seller_city, seller_state

Candidate Keys: {seller_id} **Primary Key:** seller_id

1NF: All attributes are atomic and single-valued. The table is in 1NF.

2NF: The primary key is a single attribute, so partial dependencies cannot occur. The table is in 2NF.

3NF: A transitive dependency exists: seller_id → seller_zip_code_prefix, and seller_zip_code_prefix → seller_city, seller_state. This means seller_city and seller_state are determined transitively through the zip code prefix rather than directly by seller_id. This violates 3NF, and mirrors the same transitive dependency found in the Customer relation.

Decomposition into 3NF:

Seller(seller_id, seller_zip_code_prefix) SellerZip(seller_zip_code_prefix, seller_city, seller_state)

BCNF: In Seller, seller_id is the only determinant and is a superkey. In SellerZip, seller_zip_code_prefix → seller_city, seller_state, where seller_zip_code_prefix is the primary key and a superkey. Both relations satisfy BCNF. As with the Customer decomposition, SellerZip consolidates with the existing Geolocation relation in the schema.

Final Form: BCNF (after decomposition from 3NF violation)

9. Product Category Name Translation

Relation: ProductCategoryTranslation(product_category_name, product_category_name_english)

Functional Dependencies:

- product_category_name → product_category_name_english
- product_category_name_english → product_category_name

Candidate Keys: {product_category_name}, {product_category_name_english} **Primary Key:** product_category_name

1NF: Both attributes are atomic and single-valued. There are no repeating groups. The table is in 1NF.

2NF: The primary key is a single attribute, making partial dependencies impossible. The table is in 2NF.

3NF: There are only two attributes in the relation, both of which are candidate keys. No non-key attribute exists that could create a transitive dependency. The table is in 3NF.

BCNF: Both determinants — product_category_name and product_category_name_english — are candidate keys and therefore superkeys. Every non-trivial functional dependency has a superkey as its determinant. The table is in BCNF.

Final Form: BCNF

VI. SQL Queries

Creation of tables using SQLite3 and running of sample commands from database, displaying concepts learned across duration of course:

1. List the customer ID, unique ID, zip code, city, and state for all customers from São Paulo state.

```
QUERY: Q1 — SELECT with WHERE: First 10 customers from São Paulo

SQL:
SELECT customer_id, customer_unique_id, customer_zip_code_prefix,
           customer_city, customer_state
    FROM customers
    WHERE customer_state = 'SP'
    LIMIT 10

OUTPUT:
                      customer_id                customer_unique_id customer_zip_code_prefix customer_city customer_state
8ab97904e6daea8866dbdbc4fb7aad2c 72632f0f9dd73dfee390c9b22eb56dd6                     9195   santo andre             SP
9ef432eb6251297304e76186b10a928d 7c396fd4830fd04220f754e42b4e5bff                     3149     sao paulo             SP
31ad1d1b63eb9962463f764d4e6e0c9d 299905e3934e9e181bfb2e164dd4b4f8                    18075      sorocaba             SP
```

2. How many customers are there in each state, and which states have the most customers?

```
------------------------------------------------------------
QUERY: Q2 — GROUP BY + COUNT: Number of customers per state

SQL:
SELECT customer_state, COUNT(*) AS total_customers
    FROM customers
    GROUP BY customer_state
    ORDER BY total_customers DESC
    LIMIT 10

OUTPUT:
customer_state  total_customers
            SP                3
            RS                2
            RN                1
            RJ                1
            PR                1
            GO                1
            BA                1
```

3. What is the total revenue and number of distinct orders per seller, ranked from highest to lowest revenue?

```
QUERY: Q3 — GROUP BY + SUM: Total revenue per seller (top 10)

SQL:
SELECT seller_id,
           ROUND(SUM(price), 2)              AS total_revenue,
           COUNT(DISTINCT order_id)          AS total_orders
    FROM order_items
    GROUP BY seller_id
    ORDER BY total_revenue DESC
    LIMIT 10

OUTPUT:
                         seller_id  total_revenue  total_orders
4869f7a5dfa277a7dca6462dcf3b52b2         159.90             1
7c67e1448b00f6e969d365cea6b010ab         149.99             1
8581055ce74af1daba164fdbd55a40de         147.90             1
289cdb325fb7e7f891c38608bf9e0962         118.70             1
16090f2ca825584b5a147ab24aa30c86          59.99             1
dc8798cbf453b7e0f98745e396cc5616          49.90             1
66922902710d126a0e7d26b0e3805106          45.00             1
3504c0cb71d7fa48d967e0e4c94d59d9          29.99             1
63b9ae557efed31d1f7687917d248a8d          19.90             1
2c9e548be18521d1c43cde1c582c6de8          19.90             1
```

What is the average customer review score and total number of reviews for each product category, ranked from highest to lowest average score?

```
QUERY: Q4 — Multi-table JOIN + AVG: Average review score per product category (top 10)

SQL:
SELECT t.product_category_name_english,
           ROUND(AVG(r.review_score), 2) AS avg_score,
           COUNT(r.review_id)            AS total_reviews
    FROM order_reviews r
    JOIN orders      o   ON r.order_id   = o.order_id
    JOIN order_items oi  ON o.order_id   = oi.order_id
    JOIN products    p   ON oi.product_id = p.product_id
    JOIN product_category_name_translation t
                         ON p.product_category_name = t.product_category_name
    GROUP BY t.product_category_name_english
    ORDER BY avg_score DESC
    LIMIT 10

OUTPUT:
product_category_name_english  avg_score  total_reviews
                   stationery       5.00              1
                     pet_shop       5.00              1
              office_furniture      5.00              1
                         auto       4.67              3
                    perfumery       4.00              1
                   housewares       4.00              1
                furniture_decor      1.00              1
```

5. How many orders with a status of "delivered" were placed in 2018?

```
QUERY: Q5 — WHERE + LIKE: Count of delivered orders placed in 2018

SQL:
SELECT COUNT(*) AS delivered_orders_2018
    FROM orders
    WHERE order_status = 'delivered'
      AND order_purchase_timestamp LIKE '2018%'

OUTPUT:
 delivered_orders_2018
                     3
```

6. What are the maximum, minimum, and average payment values for each payment type, and how many transactions does each payment type have?

```
QUERY: Q6 — GROUP BY + MAX/MIN/AVG: Payment statistics by payment type

SQL:
SELECT payment_type,
           ROUND(MAX(payment_value), 2) AS max_payment,
           ROUND(MIN(payment_value), 2) AS min_payment,
           ROUND(AVG(payment_value), 2) AS avg_payment,
           COUNT(*)                     AS total_transactions
    FROM order_payments
    GROUP BY payment_type
    ORDER BY avg_payment DESC

OUTPUT:
payment_type  max_payment  min_payment  avg_payment  total_transactions
      boleto       141.46        35.95        88.71                   2
 credit_card       179.12         8.34        88.03                   6
     voucher       161.42         2.00        81.71                   2
```

7. For each order, what is the order status, purchase timestamp, and the city and state of the customer who placed it?

```
QUERY: Q7 — INNER JOIN: Orders with customer city and state (first 10)

SQL:
SELECT o.order_id, o.order_status, o.order_purchase_timestamp,
          c.customer_city, c.customer_state
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    LIMIT 10

OUTPUT:
                       order_id order_status order_purchase_timestamp          customer_city customer_state
e481f51cbdc54678b7cc49136f2d6af7    delivered      2017-10-02 10:56:33              sao paulo             SP
53cdb2fc8bc7dce0b6741e2150273451    delivered      2018-07-24 20:41:37              barreiras             BA
47770eb9100c2d0c44946d9cf07ec65d    delivered      2018-08-08 08:38:49             vianopolis             GO
949d5b44dbf5de918fe9c16f97b45f8a    delivered      2017-11-18 19:28:06 sao goncalo do amarante            RN
ad21c59c0840e6cb83a9ceb5573f8159    delivered      2018-02-13 21:18:39             santo andre            SP
a4591c265e18cb1dcee52889e2d8acc3    delivered      2017-07-09 21:57:05            congonhinhas            PR
136cce7faa42fdb2cefd53fdc79a6098     invoiced      2017-04-11 12:22:08              santa rosa            RS
6514b8ad8028c9f2cc2374ded245783f    delivered      2017-05-16 13:10:30              nilopolis             RJ
76c6e866289321a7c93b82b54852dc33    delivered      2017-01-23 18:29:09            faxinalzinho            RS
e69bfb5eb88e0ed6a785585b27e16dbf    delivered      2017-07-29 11:55:02                sorocaba            SP
```

8. List the seller ID, zip code, city, and state for all sellers based in São Paulo state.

```
QUERY: Q8 — WHERE: Sellers operating from São Paulo state (first 10)

SQL:
SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state
    FROM sellers
    WHERE seller_state = 'SP'
    LIMIT 10

OUTPUT:
                         seller_id seller_zip_code_prefix           seller_city seller_state
289cdb325fb7e7f891c38608bf9e0962                   31570        belo horizonte           SP
3504c0cb71d7fa48d967e0e4c94d59d9                    9350                  maua           SP
7c67e1448b00f6e969d365cea6b010ab                    8577       itaquaquecetuba           SP
2c9e548be18521d1c43cde1c582c6de8                    8752       mogi das cruzes           SP
8581055ce74af1daba164fdbd55a40de                    7112             guarulhos           SP
16090f2ca825584b5a147ab24aa30c86                   12940               atibaia           SP
dc8798cbf453b7e0f98745e396cc5616                    5455             sao paulo           SP
4869f7a5dfa277a7dca6462dcf3b52b2                   14840               guariba           SP
63b9ae557efed31d1f7687917d248a8d                   13720 sao jose do rio pardo           SP
```

9. Which products weigh more than 5 kg, and what are their dimensions, sorted from heaviest to lightest?

```
QUERY: Q9 — WHERE + ORDER BY: Products heavier than 5 kg (top 10 by weight)

SQL:
SELECT product_id, product_category_name, product_weight_g,
          product_length_cm, product_height_cm, product_width_cm
    FROM products
    WHERE product_weight_g > 5000
    ORDER BY product_weight_g DESC
    LIMIT 10

OUTPUT:
                       product_id product_category_name  product_weight_g  product_length_cm  product_height_cm  product_width_cm
9a78fb9862b10749a117f7fc3c31f051     moveis_escritorio            9750.0               42.0               41.0              42.0
060cb19345d90064d1015407193c233d            automotivo            7150.0               65.0               10.0              65.0
```

10. For each order, how many items does it contain and what is the total freight cost, ranked from most to least expensive freight?

```
QUERY: Q10 — GROUP BY + SUM: Total freight cost per order (top 10)

SQL:
SELECT order_id,
           COUNT(order_item_id)          AS item_count,
           ROUND(SUM(freight_value), 2)  AS total_freight
    FROM order_items
    GROUP BY order_id
    ORDER BY total_freight DESC
    LIMIT 10

OUTPUT:
                          order_id  item_count  total_freight
a4591c265e18cb1dcee52889e2d8acc3           1          27.36
949d5b44dbf5de918fe9c16f97b45f8a           1          27.20
53cdb2fc8bc7dce0b6741e2150273451           1          22.76
e69bfb5eb88e0ed6a785585b27e16dbf           1          19.77
47770eb9100c2d0c44946d9cf07ec65d           1          19.22
76c6e866289321a7c93b82b54852dc33           1          16.05
136cce7faa42fdb2cefd53fdc79a6098           1          16.05
6514b8ad8028c9f2cc2374ded245783f           1          15.17
e481f51cbdc54678b7cc49136f2d6af7           1           8.72
ad21c59c0840e6cb83a9ceb5573f8159           1           8.72
```

11. Which orders have a total payment value above the overall average payment value across all transactions?

```
QUERY: Q11 — HAVING + Subquery: Orders with total payment above the overall average (top 10)

SQL:
SELECT order_id,
           ROUND(SUM(payment_value), 2) AS total_paid
    FROM order_payments
    GROUP BY order_id
    HAVING SUM(payment_value) > (SELECT AVG(payment_value) FROM order_payments)
    ORDER BY total_paid DESC
    LIMIT 10

OUTPUT:
                        order_id  total_paid
47770eb9100c2d0c44946d9cf07ec65d      179.12
a4591c265e18cb1dcee52889e2d8acc3      175.26
e69bfb5eb88e0ed6a785585b27e16dbf      169.76
53cdb2fc8bc7dce0b6741e2150273451      141.46
```

12. Which sellers have never appeared in any order item — that is, they are registered but have not fulfilled a single order?

```
QUERY: Q12 — LEFT JOIN + IS NULL: Sellers with no order items

SQL:
SELECT s.seller_id, s.seller_city, s.seller_state
    FROM sellers s
    LEFT JOIN order_items oi ON s.seller_id = oi.seller_id
    WHERE oi.seller_id IS NULL
    LIMIT 10

OUTPUT:
Empty DataFrame
Columns: [seller_id, seller_city, seller_state]
Index: []
```

13. Insert a new seller with ID "TEST_SELLER_001" located in São Paulo, then confirm the record was saved correctly.

```
QUERY: Q13 — INSERT: Add a new seller record

SQL:
INSERT INTO sellers (seller_id, seller_zip_code_prefix, seller_city, seller_state)
VALUES ('TEST_SELLER_001', '01310', 'sao paulo', 'SP');
```

```
QUERY: Q13 — Verify INSERT

SQL:
SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state
    FROM sellers
    WHERE seller_id = 'TEST_SELLER_001'

OUTPUT:
      seller_id seller_zip_code_prefix seller_city seller_state
TEST_SELLER_001                 01310   sao paulo           SP
```

14. Update the city and state of seller "TEST_SELLER_001" to Rio de Janeiro, then confirm the change was applied.

```
QUERY: Q14 — UPDATE: Change city and state of the inserted seller

SQL:
UPDATE sellers SET seller_city = 'rio de janeiro', seller_state = 'RJ'
WHERE seller_id = 'TEST_SELLER_001';
```

```
QUERY: Q14 — Verify UPDATE

SQL:
SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state
    FROM sellers
    WHERE seller_id = 'TEST_SELLER_001'

OUTPUT:
      seller_id seller_zip_code_prefix    seller_city seller_state
TEST_SELLER_001                 01310 rio de janeiro           RJ
```

15. Delete the record for seller "TEST_SELLER_001", then confirm no rows remain for that ID.

```
QUERY: Q15 — DELETE: Remove the test seller record

SQL:
DELETE FROM sellers WHERE seller_id = 'TEST_SELLER_001';
```

```
QUERY: Q15 — Verify DELETE

SQL:
SELECT COUNT(*) AS remaining_test_records
    FROM sellers
    WHERE seller_id = 'TEST_SELLER_001'

OUTPUT:
 remaining_test_records
                      0
```

16. Which product categories appear in more than one order item, and how many order items does each have?

```
QUERY: Q16 — JOIN + HAVING: Product categories with more than 1 order item (top 10)

SQL:
SELECT p.product_category_name,
          COUNT(oi.order_id) AS total_order_items
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.product_category_name
    HAVING COUNT(oi.order_id) > 1
    ORDER BY total_order_items DESC
    LIMIT 10

OUTPUT:
product_category_name  total_order_items
          automotivo                  3
```

17. What is the highest price each product has ever been sold for, and what category does it belong to, ranked from most to least expensive?

```
QUERY: Q17 — JOIN + MAX + ORDER BY: Top 10 most expensive products by unit price

SQL:
SELECT oi.product_id, p.product_category_name,
           ROUND(MAX(oi.price), 2) AS max_price
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY oi.product_id, p.product_category_name
    ORDER BY max_price DESC
    LIMIT 10

OUTPUT:
                       product_id product_category_name  max_price
aa4383b373c6aca5d8797843e5594415            automotivo     159.90
9a78fb9862b10749a117f7fc3c31f051     moveis_escritorio     149.99
060cb19345d90064d1015407193c233d            automotivo     147.90
595fac2a385ac33a80bd5114aec74eb8            perfumaria     118.70
4520766ec412348b8d4caa5e8a18c464            automotivo      59.99
a1804276d9941ac0733cfd409f5206eb                  None      49.90
d0b61bfb1de832b15ba9d266ca96e5b0             pet_shop      45.00
87285b34884572647811a353c7ac498a utilidades_domesticas      29.99
65266b2da20d04dbe00c5c2d3bb7859e             papelaria      19.90
ac1789e492dcd698c5c10b97a671243a      moveis_decoracao      19.90
```

18. On average, how many days did delivery take from purchase date to delivery date, broken down by customer state, from fastest to slowest?

```
QUERY: Q18 — JOIN + AVG + Date Arithmetic: Average delivery time per state (top 10 fastest)

SQL:
SELECT c.customer_state,
           ROUND(AVG(
               JULIANDAY(o.order_delivered_customer_date) -
               JULIANDAY(o.order_purchase_timestamp)
           ), 1) AS avg_delivery_days
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.order_delivered_customer_date IS NOT NULL
    GROUP BY c.customer_state
    ORDER BY avg_delivery_days ASC
    LIMIT 10

OUTPUT:
customer_state  avg_delivery_days
            GO                9.4
            RS                9.8
            SP                9.8
            RJ               10.0
            RN               13.2
            BA               13.8
            PR               16.5
```

19. What is the combined list of all unique cities that appear in either the customers table or the sellers table?

```
QUERY: Q19 — SET OPERATION (UNION): Cities from customers OR sellers (first 15)

SQL:
SELECT customer_city AS city, 'customer' AS source FROM customers
    UNION
    SELECT seller_city   AS city, 'seller'   AS source FROM sellers
    ORDER BY city
    LIMIT 15

OUTPUT:
                   city   source
                atibaia   seller
             barreiras customer
         belo horizonte   seller
           congonhinhas customer
           faxinalzinho customer
                guariba   seller
              guarulhos   seller
        itaquaquecetuba   seller
                  maua   seller
        mogi das cruzes   seller
              nilopolis customer
             santa rosa customer
            santo andre customer
sao goncalo do amarante customer
  sao jose do rio pardo   seller
```

20. Which cities appear in both the customers table and the sellers table?

```
QUERY: Q20 — SET OPERATION (INTERSECT): Cities with both customers AND sellers (first 15)

SQL:
SELECT customer_city AS city FROM customers
    INTERSECT
    SELECT seller_city   AS city FROM sellers
    ORDER BY city
    LIMIT 15

OUTPUT:
     city
sao paulo
```

21. Which customer cities have no seller registered in them?

```
QUERY: Q21 — SET OPERATION (EXCEPT): Customer cities with no registered seller (first 15)

SQL:
SELECT customer_city AS city FROM customers
    EXCEPT
    SELECT seller_city   AS city FROM sellers
    ORDER BY city
    LIMIT 15

OUTPUT:
                   city
              barreiras
           congonhinhas
           faxinalzinho
              nilopolis
             santa rosa
            santo andre
sao goncalo do amarante
               sorocaba
            vianopolis
```

22. Which customers have placed more than one order, and how many orders has each placed?

```
QUERY: Q22 — Subquery + HAVING: Repeat buyers with more than one order (top 10)

SQL:
SELECT c.customer_unique_id,
           COUNT(o.order_id) AS order_count
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_unique_id
    HAVING COUNT(o.order_id) > 1
    ORDER BY order_count DESC
    LIMIT 10

OUTPUT:
Empty DataFrame
Columns: [customer_unique_id, order_count]
Index: []
```

23. Within a single transaction, insert two sellers, roll back the second insertion to a savepoint, then commit — which sellers survive?

```
QUERY: Q23 — TCL: BEGIN / SAVEPOINT / ROLLBACK TO / COMMIT

SQL:
BEGIN;
INSERT INTO sellers VALUES ('TXN_TEST_A', '10000', 'brasilia',  'DF');
SAVEPOINT sp1;
INSERT INTO sellers VALUES ('TXN_TEST_B', '20000', 'manaus',    'AM');
ROLLBACK TO sp1;
COMMIT;
```

```
QUERY: Q23 — Verify TCL: Only TXN_TEST_A should exist (TXN_TEST_B was rolled back)

SQL:
SELECT seller_id, seller_city, seller_state
    FROM sellers
    WHERE seller_id IN ('TXN_TEST_A', 'TXN_TEST_B')

OUTPUT:
 seller_id seller_city seller_state
TXN_TEST_A    brasilia           DF
```

24. Which states had the most orders delivered after their estimated delivery date?

```
QUERY: Q24 — WHERE + Date Comparison + JOIN: Late deliveries per state (top 10)

SQL:
SELECT c.customer_state,
           COUNT(*) AS late_deliveries
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.order_delivered_customer_date > o.order_estimated_delivery_date
      AND o.order_delivered_customer_date IS NOT NULL
    GROUP BY c.customer_state
    ORDER BY late_deliveries DESC
    LIMIT 10

OUTPUT:
Empty DataFrame
Columns: [customer_state, late_deliveries]
Index: []
```

25. For each delivered order, what was the total amount paid and what review score did the customer leave, joined across customer, payment, and review data?

```
QUERY: Q25 — 4-table JOIN (incl. LEFT JOIN): Full order summary with payment and review (first 10)

SQL:
SELECT o.order_id,
           c.customer_state,
           o.order_status,
           ROUND(SUM(p.payment_value), 2) AS total_paid,
           r.review_score
    FROM orders o
    JOIN      customers     c  ON o.customer_id = c.customer_id
    JOIN      order_payments p  ON o.order_id   = p.order_id
    LEFT JOIN order_reviews  r  ON o.order_id   = r.order_id
    GROUP BY o.order_id, c.customer_state, o.order_status, r.review_score
    LIMIT 10

OUTPUT:
                         order_id customer_state order_status  total_paid  review_score
47770eb9100c2d0c44946d9cf07ec65d             GO    delivered      179.12             5
53cdb2fc8bc7dce0b6741e2150273451             BA    delivered      141.46             4
6514b8ad8028c9f2cc2374ded245783f             RJ    delivered       75.16             5
76c6e866289321a7c93b82b54852dc33             RS    delivered       35.95             1
949d5b44dbf5de918fe9c16f97b45f8a             RN    delivered       72.20             5
a4591c265e18cb1dcee52889e2d8acc3             PR    delivered      175.26             4
e481f51cbdc54678b7cc49136f2d6af7             SP    delivered       20.12             4
e69bfb5eb88e0ed6a785585b27e16dbf             SP    delivered      169.76             5
```

SQL CODE:

```
CREATE TABLE customers (
    customer_id TEXT,
    customer_unique_id TEXT,
    customer_zip_code_prefix TEXT,
    customer_city TEXT,
    customer_state TEXT
);

CREATE TABLE orders (
    order_id TEXT,
    customer_id TEXT,
    order_status TEXT,
    order_purchase_timestamp TEXT,
    order_approved_at TEXT,
    order_delivered_carrier_date TEXT,
    order_delivered_customer_date TEXT,
    order_estimated_delivery_date TEXT
);

CREATE TABLE order_items (
    order_id TEXT,
    order_item_id TEXT,
    product_id TEXT,
    seller_id TEXT,
    shipping_limit_date TEXT,
    price REAL,
    freight_value REAL
```

```
);

CREATE TABLE order_payments (
    order_id TEXT,
    payment_sequential INTEGER,
    payment_type TEXT,
    payment_installments INTEGER,
    payment_value REAL
);

CREATE TABLE order_reviews (
    review_id TEXT,
    order_id TEXT,
    review_score INTEGER,
    review_comment_title TEXT,
    review_comment_message TEXT,
    review_creation_date TEXT,
    review_answer_timestamp TEXT
);

CREATE TABLE products (
    product_id TEXT,
    product_category_name TEXT,
    product_name_lenght REAL,
    product_description_lenght REAL,
    product_photos_qty REAL,
    product_weight_g REAL,
```

```
    product_length_cm REAL,
    product_height_cm REAL,
    product_width_cm REAL
);

CREATE TABLE sellers (
    seller_id TEXT,
    seller_zip_code_prefix TEXT,
    seller_city TEXT,
    seller_state TEXT
);

CREATE TABLE product_category_name_translation (
    product_category_name TEXT,
    product_category_name_english TEXT
);

CREATE TABLE geolocation (
    geolocation_zip_code_prefix TEXT,
    geolocation_lat REAL,
    geolocation_lng REAL,
    geolocation_city TEXT,
    geolocation_state TEXT
);

INSERT INTO customers VALUES ('41ce2a54c0b03bf3443c3d931a367089',
'3a653a41f6f9fc3d2a113cf8398680e8', '75265', 'vianopolis', 'GO');
```

INSERT INTO customers VALUES ('f54a9f0e6b351c431402b8461ea51999',
'39382392765b6dc74812866ee5ee92a7', '99655', 'faxinalzinho', 'RS');

INSERT INTO customers VALUES ('f88197465ea7920adcdbec7375364d82',
'7c142cf63193a1473d2e66489a9ae977', '59296', 'sao goncalo do amarante', 'RN');

INSERT INTO customers VALUES ('8ab97904e6daea8866dbdbc4fb7aad2c',
'72632f0f9dd73dfee390c9b22eb56dd6', '9195', 'santo andre', 'SP');

INSERT INTO customers VALUES ('503740e9ca751ccdda7ba28e9ab8f608',
'80bb27c7c16e8f973207a5086ab329e2', '86320', 'congonhinhas', 'PR');

INSERT INTO customers VALUES ('9ef432eb6251297304e76186b10a928d',
'7c396fd4830fd04220f754e42b4e5bff', '3149', 'sao paulo', 'SP');

INSERT INTO customers VALUES ('ed0271e0b7da060a393796590e7b737a',
'36edbb3fb164b1f16485364b6fb04c73', '98900', 'santa rosa', 'RS');

INSERT INTO customers VALUES ('b0830fb4747a6c6d20dea0b8c802d7ef',
'af07308b275d755c9edb36a90c618231', '47813', 'barreiras', 'BA');

INSERT INTO customers VALUES ('9bdf08b4b3b52b5526ff42d37d47f222',
'932afa1e708222e5821dac9cd5db4cae', '26525', 'nilopolis', 'RJ');

INSERT INTO customers VALUES ('31ad1d1b63eb9962463f764d4e6e0c9d',
'299905e3934e9e181bfb2e164dd4b4f8', '18075', 'sorocaba', 'SP');

INSERT INTO orders VALUES ('e481f51cbdc54678b7cc49136f2d6af7',
'9ef432eb6251297304e76186b10a928d', 'delivered', '2017-10-02 10:56:33', '2017-10-02 11:07:15', '2017-10-04 19:55:00', '2017-10-10 21:25:13', '2017-10-18 00:00:00');

INSERT INTO orders VALUES ('53cdb2fc8bc7dce0b6741e2150273451',
'b0830fb4747a6c6d20dea0b8c802d7ef', 'delivered', '2018-07-24 20:41:37', '2018-07-26 03:24:27', '2018-07-26 14:31:00', '2018-08-07 15:27:45', '2018-08-13 00:00:00');

INSERT INTO orders VALUES ('47770eb9100c2d0c44946d9cf07ec65d',
'41ce2a54c0b03bf3443c3d931a367089', 'delivered', '2018-08-08 08:38:49', '2018-08-08 08:55:23', '2018-08-08 13:50:00', '2018-08-17 18:06:29', '2018-09-04 00:00:00');

INSERT INTO orders VALUES ('949d5b44dbf5de918fe9c16f97b45f8a',
'f88197465ea7920adcdbec7375364d82', 'delivered', '2017-11-18 19:28:06', '2017-11-18 19:45:59', '2017-11-22 13:39:59', '2017-12-02 00:28:42', '2017-12-15 00:00:00');

INSERT INTO orders VALUES ('ad21c59c0840e6cb83a9ceb5573f8159',
'8ab97904e6daea8866dbdbc4fb7aad2c', 'delivered', '2018-02-13 21:18:39', '2018-02-13 22:20:29', '2018-02-14 19:46:34', '2018-02-16 18:17:02', '2018-02-26 00:00:00');

INSERT INTO orders VALUES ('a4591c265e18cb1dcee52889e2d8acc3',
'503740e9ca751ccdda7ba28e9ab8f608', 'delivered', '2017-07-09 21:57:05', '2017-07-09 22:10:13', '2017-07-11

14:58:04', '2017-07-26 10:57:55', '2017-08-01 00:00:00');

INSERT INTO orders VALUES ('136cce7faa42fdb2cefd53fdc79a6098', 'ed0271e0b7da060a393796590e7b737a', 'invoiced', '2017-04-11 12:22:08', '2017-04-13 13:25:17', NULL, NULL, '2017-05-09 00:00:00');

INSERT INTO orders VALUES ('6514b8ad8028c9f2cc2374ded245783f', '9bdf08b4b3b52b5526ff42d37d47f222', 'delivered', '2017-05-16 13:10:30', '2017-05-16 13:22:11', '2017-05-22 10:07:46', '2017-05-26 12:55:51', '2017-06-07 00:00:00');

INSERT INTO orders VALUES ('76c6e866289321a7c93b82b54852dc33', 'f54a9f0e6b351c431402b8461ea51999', 'delivered', '2017-01-23 18:29:09', '2017-01-25 02:50:47', '2017-01-26 14:16:31', '2017-02-02 14:08:10', '2017-03-06 00:00:00');

INSERT INTO orders VALUES ('e69bfb5eb88e0ed6a785585b27e16dbf', '31ad1d1b63eb9962463f764d4e6e0c9d', 'delivered', '2017-07-29 11:55:02', '2017-07-29 12:05:32', '2017-08-10 19:45:24', '2017-08-16 17:14:30', '2017-08-23 00:00:00');

INSERT INTO order_items VALUES ('136cce7faa42fdb2cefd53fdc79a6098', '1', 'a1804276d9941ac0733cfd409f5206eb', 'dc8798cbf453b7e0f98745e396cc5616', '2017-04-19 13:25:17', 49.9, 16.05);

INSERT INTO order_items VALUES ('47770eb9100c2d0c44946d9cf07ec65d', '1', 'aa4383b373c6aca5d8797843e5594415', '4869f7a5dfa277a7dca6462dcf3b52b2', '2018-08-13 08:55:23', 159.9, 19.22);

INSERT INTO order_items VALUES ('53cdb2fc8bc7dce0b6741e2150273451', '1', '595fac2a385ac33a80bd5114aec74eb8', '289cdb325fb7e7f891c38608bf9e0962', '2018-07-30 03:24:27', 118.7, 22.76);

INSERT INTO order_items VALUES ('6514b8ad8028c9f2cc2374ded245783f', '1', '4520766ec412348b8d4caa5e8a18c464', '16090f2ca825584b5a147ab24aa30c86', '2017-05-22 13:22:11', 59.99, 15.17);

INSERT INTO order_items VALUES ('76c6e866289321a7c93b82b54852dc33', '1', 'ac1789e492dcd698c5c10b97a671243a', '63b9ae557efed31d1f7687917d248a8d', '2017-01-27 18:29:09', 19.9, 16.05);

INSERT INTO order_items VALUES ('949d5b44dbf5de918fe9c16f97b45f8a', '1', 'd0b61bfb1de832b15ba9d266ca96e5b0', '66922902710d126a0e7d26b0e3805106', '2017-11-23 19:45:59', 45.0, 27.2);

INSERT INTO order_items VALUES ('a4591c265e18cb1dcee52889e2d8acc3', '1', '060cb19345d90064d1015407193c233d', '8581055ce74af1daba164fdbd55a40de', '2017-07-13 22:10:13', 147.9, 27.36);

INSERT INTO order_items VALUES ('ad21c59c0840e6cb83a9ceb5573f8159', '1', '65266b2da20d04dbe00c5c2d3bb7859e', '2c9e548be18521d1c43cde1c582c6de8', '2018-02-19 20:31:37', 19.9,

8.72);

INSERT INTO order_items VALUES ('e481f51cbdc54678b7cc49136f2d6af7', '1',
'87285b34884572647811a353c7ac498a', '3504c0cb71d7fa48d967e0e4c94d59d9', '2017-10-06 11:07:15', 29.99,
8.72);

INSERT INTO order_items VALUES ('e69bfb5eb88e0ed6a785585b27e16dbf', '1',
'9a78fb9862b10749a117f7fc3c31f051', '7c67e1448b00f6e969d365cea6b010ab', '2017-08-11 12:05:32', 149.99,
19.77);

INSERT INTO order_payments VALUES ('76c6e866289321a7c93b82b54852dc33', 1, 'boleto', 1, 35.95);

INSERT INTO order_payments VALUES ('53cdb2fc8bc7dce0b6741e2150273451', 1, 'boleto', 1, 141.46);

INSERT INTO order_payments VALUES ('e481f51cbdc54678b7cc49136f2d6af7', 1, 'credit_card', 1, 18.12);

INSERT INTO order_payments VALUES ('47770eb9100c2d0c44946d9cf07ec65d', 1, 'credit_card', 3, 179.12);

INSERT INTO order_payments VALUES ('6514b8ad8028c9f2cc2374ded245783f', 1, 'credit_card', 3, 75.16);

INSERT INTO order_payments VALUES ('e69bfb5eb88e0ed6a785585b27e16dbf', 2, 'voucher', 1, 161.42);

INSERT INTO order_payments VALUES ('949d5b44dbf5de918fe9c16f97b45f8a', 1, 'credit_card', 1, 72.2);

INSERT INTO order_payments VALUES ('a4591c265e18cb1dcee52889e2d8acc3', 1, 'credit_card', 6, 175.26);

INSERT INTO order_payments VALUES ('e481f51cbdc54678b7cc49136f2d6af7', 3, 'voucher', 1, 2.0);

INSERT INTO order_payments VALUES ('e69bfb5eb88e0ed6a785585b27e16dbf', 1, 'credit_card', 1, 8.34);

INSERT INTO order_reviews VALUES ('e73b67b67587f7644d5bd1a52deb1b01',
'47770eb9100c2d0c44946d9cf07ec65d', 5, NULL, NULL, '2018-08-18 00:00:00', '2018-08-22 19:07:58');

INSERT INTO order_reviews VALUES ('e07549ef5311abcc92ba1784b093fb56',
'136cce7faa42fdb2cefd53fdc79a6098', 2, NULL, 'fiquei triste por n ter me atendido.', '2017-05-13 00:00:00',
'2017-05-13 20:25:42');

INSERT INTO order_reviews VALUES ('8d5266042046a06655c8db133d120ba5',
'53cdb2fc8bc7dce0b6741e2150273451', 4, 'Muito boa a loja', 'Muito bom o produto.', '2018-08-08 00:00:00',
'2018-08-08 18:37:50');

INSERT INTO order_reviews VALUES ('a54f0611adc9ed256b57ede6b6eb5114',
'e481f51cbdc54678b7cc49136f2d6af7', 4, NULL, NULL, '2017-10-11 00:00:00', '2017-10-12 03:43:48');

INSERT INTO order_reviews VALUES ('07d67dd06ed5f88bef11ef6b464e79ae',
'6514b8ad8028c9f2cc2374ded245783f', 5, NULL, NULL, '2017-05-27 00:00:00', '2017-05-28 02:59:57');

INSERT INTO order_reviews VALUES ('e50934924e227544ba8246aeb3770dd4', 'ad21c59c0840e6cb83a9ceb5573f8159', 5, NULL, NULL, '2018-02-17 00:00:00', '2018-02-18 13:02:51');

INSERT INTO order_reviews VALUES ('fc4af8aea8ec3f1a3cd181d3d0cadbd5', '76c6e866289321a7c93b82b54852dc33', 1, NULL, NULL, '2017-02-03 00:00:00', '2017-02-05 01:58:35');

INSERT INTO order_reviews VALUES ('abc5655186d40772bd6e410420e6a3ed', 'e69bfb5eb88e0ed6a785585b27e16dbf', 5, NULL, NULL, '2017-08-17 00:00:00', '2017-08-18 01:47:32');

INSERT INTO order_reviews VALUES ('359d03e676b3c069f62cadba8dd3f6e8', '949d5b44dbf5de918fe9c16f97b45f8a', 5, NULL, 'O produto foi exatamente o que eu esperava.', '2017-12-03 00:00:00', '2017-12-05 19:21:58');

INSERT INTO order_reviews VALUES ('89b738e70a1ce346db29a20fb2910161', 'a4591c265e18cb1dcee52889e2d8acc3', 4, NULL, NULL, '2017-07-27 00:00:00', '2017-07-27 22:48:30');

INSERT INTO products VALUES ('87285b34884572647811a353c7ac498a', 'utilidades_domesticas', 40.0, 268.0, 4.0, 500.0, 19.0, 8.0, 13.0);

INSERT INTO products VALUES ('ac1789e492dcd698c5c10b97a671243a', 'moveis_decoracao', 41.0, 432.0, 2.0, 300.0, 35.0, 35.0, 15.0);

INSERT INTO products VALUES ('4520766ec412348b8d4caa5e8a18c464', 'automotivo', 59.0, 956.0, 1.0, 50.0, 16.0, 16.0, 17.0);

INSERT INTO products VALUES ('65266b2da20d04dbe00c5c2d3bb7859e', 'papelaria', 38.0, 316.0, 4.0, 250.0, 51.0, 15.0, 15.0);

INSERT INTO products VALUES ('aa4383b373c6aca5d8797843e5594415', 'automotivo', 46.0, 232.0, 1.0, 420.0, 24.0, 19.0, 21.0);

INSERT INTO products VALUES ('060cb19345d90064d1015407193c233d', 'automotivo', 49.0, 608.0, 1.0, 7150.0, 65.0, 10.0, 65.0);

INSERT INTO products VALUES ('595fac2a385ac33a80bd5114aec74eb8', 'perfumaria', 29.0, 178.0, 1.0, 400.0, 19.0, 13.0, 19.0);

INSERT INTO products VALUES ('a1804276d9941ac0733cfd409f5206eb', NULL, NULL, NULL, NULL, 600.0, 35.0, 35.0, 15.0);

INSERT INTO products VALUES ('9a78fb9862b10749a117f7fc3c31f051', 'moveis_escritorio', 45.0, 527.0, 1.0, 9750.0, 42.0, 41.0, 42.0);

INSERT INTO products VALUES ('d0b61bfb1de832b15ba9d266ca96e5b0', 'pet_shop', 59.0, 468.0, 3.0, 450.0, 30.0, 10.0, 20.0);

INSERT INTO sellers VALUES ('289cdb325fb7e7f891c38608bf9e0962', '31570', 'belo horizonte', 'SP');

INSERT INTO sellers VALUES ('3504c0cb71d7fa48d967e0e4c94d59d9', '9350', 'maua', 'SP');

INSERT INTO sellers VALUES ('7c67e1448b00f6e969d365cea6b010ab', '8577', 'itaquaquecetuba', 'SP');

INSERT INTO sellers VALUES ('2c9e548be18521d1c43cde1c582c6de8', '8752', 'mogi das cruzes', 'SP');

INSERT INTO sellers VALUES ('8581055ce74af1daba164fdbd55a40de', '7112', 'guarulhos', 'SP');

INSERT INTO sellers VALUES ('16090f2ca825584b5a147ab24aa30c86', '12940', 'atibaia', 'SP');

INSERT INTO sellers VALUES ('dc8798cbf453b7e0f98745e396cc5616', '5455', 'sao paulo', 'SP');

INSERT INTO sellers VALUES ('4869f7a5dfa277a7dca6462dcf3b52b2', '14840', 'guariba', 'SP');

INSERT INTO sellers VALUES ('66922902710d126a0e7d26b0e3805106', '31842', 'belo horizonte', 'MG');

INSERT INTO sellers VALUES ('63b9ae557efed31d1f7687917d248a8d', '13720', 'sao jose do rio pardo', 'SP');

INSERT INTO product_category_name_translation VALUES ('automotivo', 'auto');

INSERT INTO product_category_name_translation VALUES ('moveis_decoracao', 'furniture_decor');

INSERT INTO product_category_name_translation VALUES ('perfumaria', 'perfumery');

INSERT INTO product_category_name_translation VALUES ('utilidades_domesticas', 'housewares');

INSERT INTO product_category_name_translation VALUES ('papelaria', 'stationery');

INSERT INTO product_category_name_translation VALUES ('pet_shop', 'pet_shop');

INSERT INTO product_category_name_translation VALUES ('moveis_escritorio', 'office_furniture');

INSERT INTO geolocation VALUES ('1037', -23.54562128115268, -46.63929204800168, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1046', -23.54608112703553, -46.64482029837157, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1046', -23.54612896641469, -46.64295148361138, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1041', -23.54439216486810, -46.63949930627844, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1035', -23.54157796171149, -46.64160722329613, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1012', -23.54776230336426, -46.63536053788448, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1047', -23.54627311241268, -46.64122516971552, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1013', -23.54692320843672, -46.63426369649150, 'sao paulo', 'SP');

INSERT INTO geolocation VALUES ('1029', -23.54376905576913, -46.63427784085132, 'sao paulo', 'SP');

```
INSERT INTO geolocation VALUES ('1011', -23.54763955032063, -46.63603162315495, 'sao paulo', 'SP');


SELECT customer_id, customer_unique_id, customer_zip_code_prefix,
    customer_city, customer_state
FROM customers
WHERE customer_state = 'SP'
LIMIT 10;


SELECT customer_state, COUNT(*) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC
LIMIT 10;


SELECT seller_id,
    ROUND(SUM(price), 2)     AS total_revenue,
    COUNT(DISTINCT order_id) AS total_orders
FROM order_items
GROUP BY seller_id
ORDER BY total_revenue DESC
LIMIT 10;


SELECT t.product_category_name_english,
    ROUND(AVG(r.review_score), 2) AS avg_score,
    COUNT(r.review_id)            AS total_reviews
FROM order_reviews r
JOIN orders      o  ON r.order_id    = o.order_id
```

```
JOIN order_items oi ON o.order_id    = oi.order_id

JOIN products    p  ON oi.product_id = p.product_id

JOIN product_category_name_translation t ON p.product_category_name = t.product_category_name

GROUP BY t.product_category_name_english

ORDER BY avg_score DESC

LIMIT 10;


SELECT COUNT(*) AS delivered_orders_2018

FROM orders

WHERE order_status = 'delivered'

  AND order_purchase_timestamp LIKE '2018%';


SELECT payment_type,

       ROUND(MAX(payment_value), 2) AS max_payment,

       ROUND(MIN(payment_value), 2) AS min_payment,

       ROUND(AVG(payment_value), 2) AS avg_payment,

       COUNT(*)                     AS total_transactions

FROM order_payments

GROUP BY payment_type

ORDER BY avg_payment DESC;


SELECT o.order_id, o.order_status, o.order_purchase_timestamp,

       c.customer_city, c.customer_state

FROM orders o

JOIN customers c ON o.customer_id = c.customer_id

LIMIT 10;
```

```
SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state
FROM sellers
WHERE seller_state = 'SP'
LIMIT 10;

SELECT product_id, product_category_name, product_weight_g,
    product_length_cm, product_height_cm, product_width_cm
FROM products
WHERE product_weight_g > 5000
ORDER BY product_weight_g DESC
LIMIT 10;

SELECT order_id,
    COUNT(order_item_id)        AS item_count,
    ROUND(SUM(freight_value), 2) AS total_freight
FROM order_items
GROUP BY order_id
ORDER BY total_freight DESC
LIMIT 10;

SELECT order_id,
    ROUND(SUM(payment_value), 2) AS total_paid
FROM order_payments
GROUP BY order_id
HAVING SUM(payment_value) > (SELECT AVG(payment_value) FROM order_payments)
ORDER BY total_paid DESC
LIMIT 10;
```

```
SELECT s.seller_id, s.seller_city, s.seller_state

FROM sellers s

LEFT JOIN order_items oi ON s.seller_id = oi.seller_id

WHERE oi.seller_id IS NULL

LIMIT 10;


INSERT INTO sellers (seller_id, seller_zip_code_prefix, seller_city, seller_state)

VALUES ('TEST_SELLER_001', '01310', 'sao paulo', 'SP');


SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state

FROM sellers

WHERE seller_id = 'TEST_SELLER_001';


UPDATE sellers

SET seller_city = 'rio de janeiro', seller_state = 'RJ'

WHERE seller_id = 'TEST_SELLER_001';


SELECT seller_id, seller_zip_code_prefix, seller_city, seller_state

FROM sellers

WHERE seller_id = 'TEST_SELLER_001';


DELETE FROM sellers

WHERE seller_id = 'TEST_SELLER_001';


SELECT COUNT(*) AS remaining_test_records

FROM sellers
```

```
WHERE seller_id = 'TEST_SELLER_001';


SELECT p.product_category_name,
    COUNT(oi.order_id) AS total_order_items
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category_name
HAVING COUNT(oi.order_id) > 1
ORDER BY total_order_items DESC
LIMIT 10;


SELECT oi.product_id, p.product_category_name,
    ROUND(MAX(oi.price), 2) AS max_price
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY oi.product_id, p.product_category_name
ORDER BY max_price DESC
LIMIT 10;


SELECT c.customer_state,
    ROUND(AVG(
        JULIANDAY(o.order_delivered_customer_date) -
        JULIANDAY(o.order_purchase_timestamp)
    ), 1) AS avg_delivery_days
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
```

```
GROUP BY c.customer_state
ORDER BY avg_delivery_days ASC
LIMIT 10;

SELECT customer_city AS city, 'customer' AS source FROM customers
UNION
SELECT seller_city AS city, 'seller' AS source FROM sellers
ORDER BY city
LIMIT 15;

SELECT customer_city AS city FROM customers
INTERSECT
SELECT seller_city AS city FROM sellers
ORDER BY city
LIMIT 15;

SELECT customer_city AS city FROM customers
EXCEPT
SELECT seller_city AS city FROM sellers
ORDER BY city
LIMIT 15;

SELECT c.customer_unique_id,
    COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id
```

```
HAVING COUNT(o.order_id) > 1
ORDER BY order_count DESC
LIMIT 10;


BEGIN;
INSERT INTO sellers VALUES ('TXN_TEST_A', '10000', 'brasilia', 'DF');
SAVEPOINT sp1;
INSERT INTO sellers VALUES ('TXN_TEST_B', '20000', 'manaus', 'AM');
ROLLBACK TO sp1;
COMMIT;


SELECT seller_id, seller_city, seller_state
FROM sellers
WHERE seller_id IN ('TXN_TEST_A', 'TXN_TEST_B');


DELETE FROM sellers WHERE seller_id = 'TXN_TEST_A';


SELECT c.customer_state,
       COUNT(*) AS late_deliveries
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date > o.order_estimated_delivery_date
  AND o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY late_deliveries DESC
LIMIT 10;
```

```
SELECT o.order_id,
    c.customer_state,
    o.order_status,
    ROUND(SUM(p.payment_value), 2) AS total_paid,
    r.review_score
FROM orders o
JOIN    customers    c ON o.customer_id = c.customer_id
JOIN    order_payments p ON o.order_id   = p.order_id
LEFT JOIN order_reviews  r ON o.order_id   = r.order_id
GROUP BY o.order_id, c.customer_state, o.order_status, r.review_score
LIMIT 10;
```

VI. Project demonstration

Python Libraries (imported)

- **streamlit** — The main web framework used to build the interactive dashboard UI (buttons, dropdowns, sidebars, forms, etc.)
- **pandas** — Used for reading CSV files, executing SQL queries into DataFrames, and displaying tabular data
- **sqlite3** — Python's built-in SQLite interface; used to create and manage an in-memory relational database
- **os** — Python's built-in module; used to list files in a folder and check if a path exists
- **matplotlib.pyplot** — Used to generate bar charts in the "Plot" operation
- **itertools.combinations** — Python's built-in utility; used in the normalization logic to generate all subsets of attributes for candidate key finding

Database

- **SQLite (in-memory)** — The project runs a full relational database entirely in RAM (sqlite3.connect(':memory:')), supporting DML (INSERT, UPDATE, DELETE), JOINs, GROUP BY, aggregations, and TCL (transactions, savepoints, rollback, commit)

Data Format

- **CSV files** — Input data is loaded from .csv files in a user-specified folder, which are then imported as tables into the SQLite database via pandas.read_csv + DataFrame.to_sql

Description of project:

Project Overview

It is a SQL Analytics dashboard - a web app created with Streamlit as a practical demonstration of fundamental concepts of Database Management Systems (DBMS) discussed during the course.

The underlying data is the Olist Brazilian E-Commerce data, a realistic set of data of a Brazilian online market. It contains 9 related CSV tables of customers, orders, order item, payment, review, product, seller, geolocation, and product category translation - a rich and normalized relational schema that resembles that of a real production database. These CSVs are then loaded into an in-memory SQLite database to create a virtual relational database at runtime and make it appear like a real database.

What the Application Demonstrates

The dashboard is structured around a collection of operations each of which corresponds to a DBMS course topic:

Basic Querying and DML- Basic SQL users have the ability to show records of any table (e.g. see the customers or orders), manipulate rows by column values, and do INSERT, UPDATE and DELETE operations - all the basics of SQL Data Manipulation Language taught in the course.

Aggregation & Group By — A specially designed module allows users to select any table, an aggregate operation (SUM, AVG, COUNT, MAX, MIN), and group the results - a direct expression of the aggregation possibilities of SQL using actual order and payment data.

Joins - Any two tables may be joined on a common column, which on this data would be naturally translated into useful queries such as joining orders by customers, or products by order items.

Set Operations — The application uses Union, Intersection, Difference, and Symmetric Difference with sets of table columns to demonstrate the principles of relational algebra with the help of overlapping attributes of the dataset.

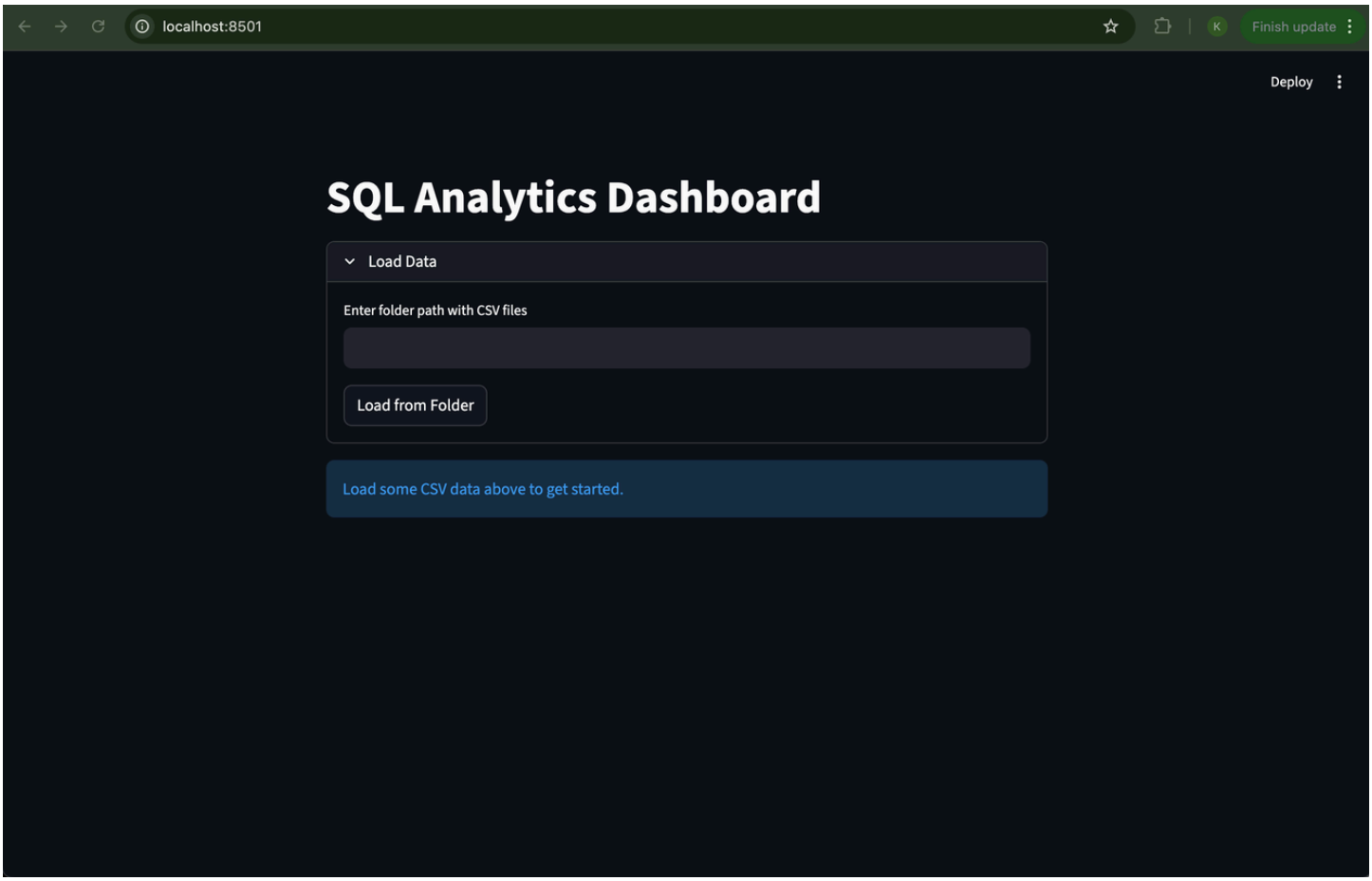
Transaction Control (TCL) — A complete TCL workflow is provided: BEGIN, SAVEpoint, ROLLback to savepoint, full ROLLback and COMMIT. This allows the users to simulate transactional behaviour on the Olist data and see pending and committed states.

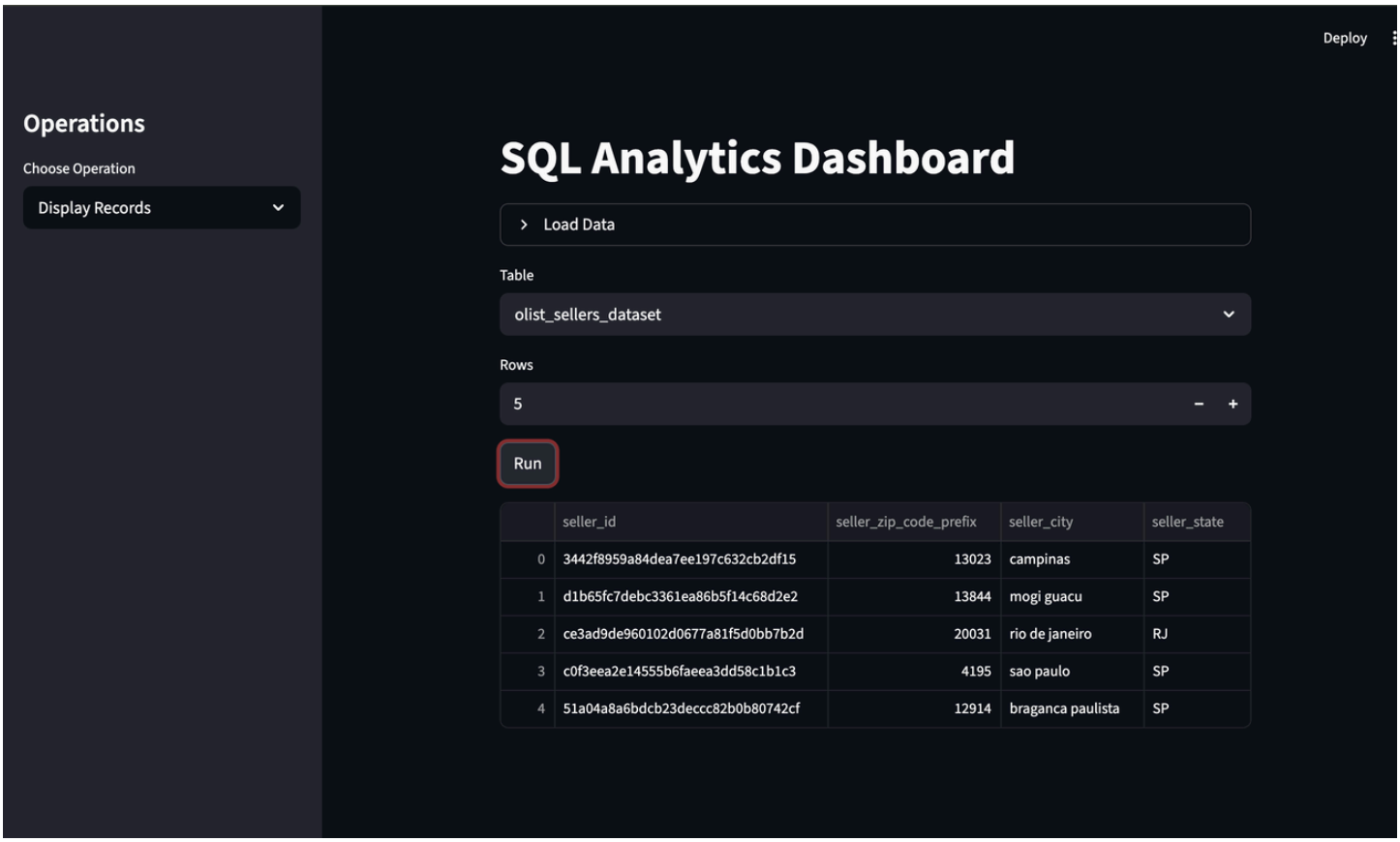
Normalization Calculator - - maybe the most scholarly of modules. The tool lets users choose any table in the dataset (e.g., the orders or products table), define functional dependencies, and the tool will automatically compute candidate keys, detect 2NF or 3NF violations (partial and transitive dependencies), decompose the schema into compliant relations, and check dependency preservation all custom-written in Python.

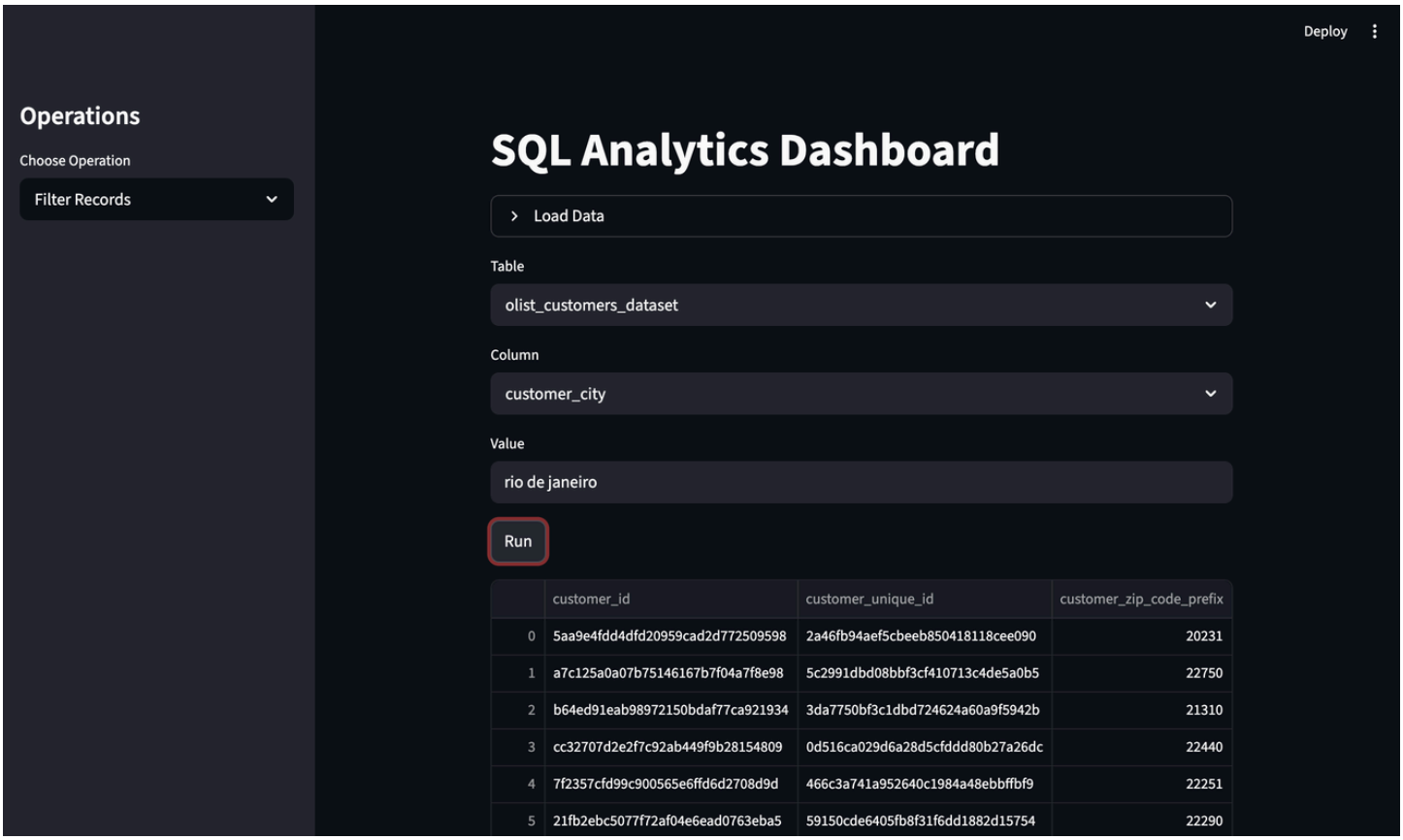
Data Visualization — A plotting option provides bar charts based on the outcome of Group By or Join commands to provide a visual analytics overlay to the data that is being queried in Olist.

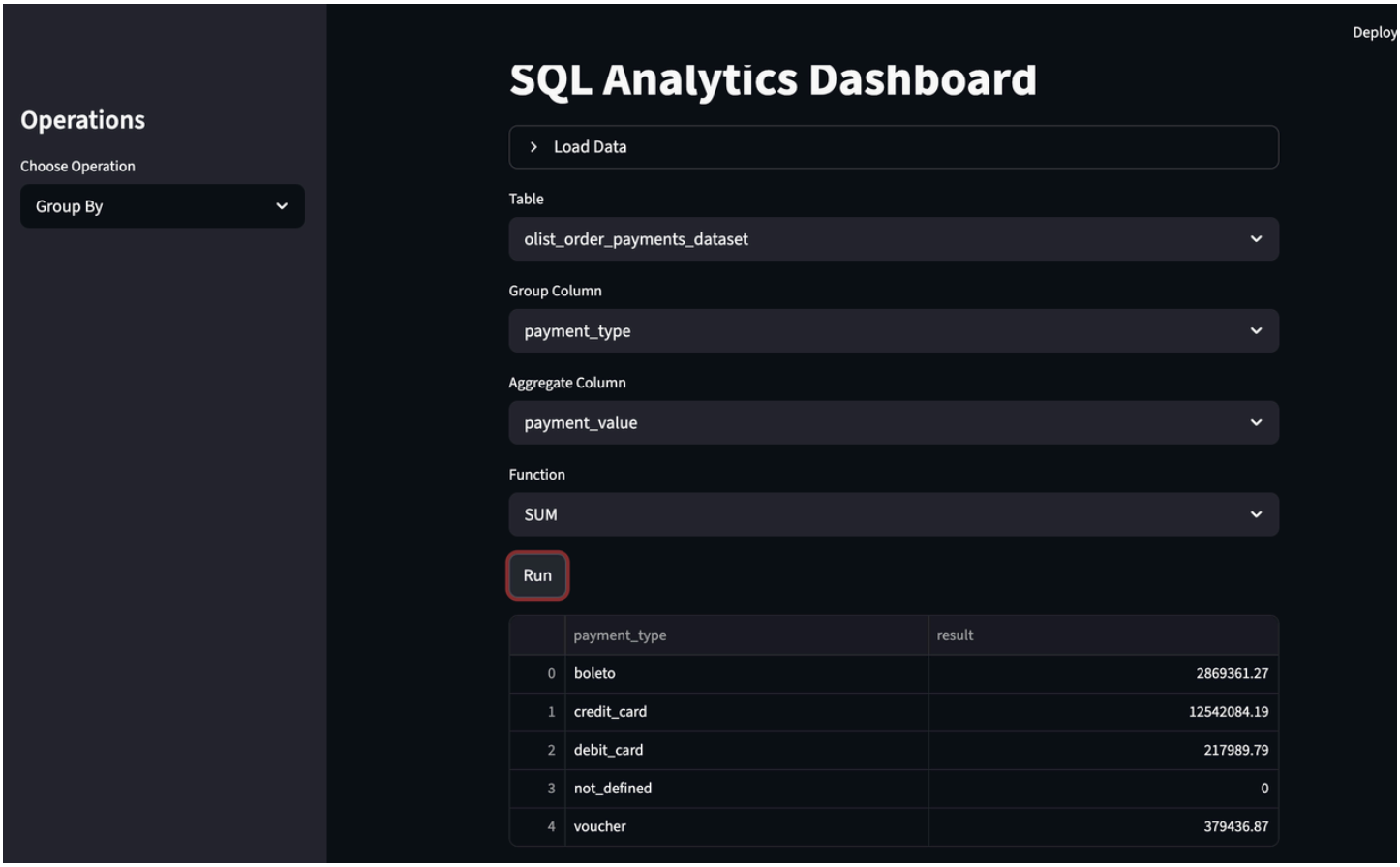
The project, in other words, is a real-world substrate to the DBMS theory, in the sense that it uses the Olist e-commerce database as a real-world object to operationalize the abstract concepts of normalization, transactions and relational algebra into a concrete, interactive object.

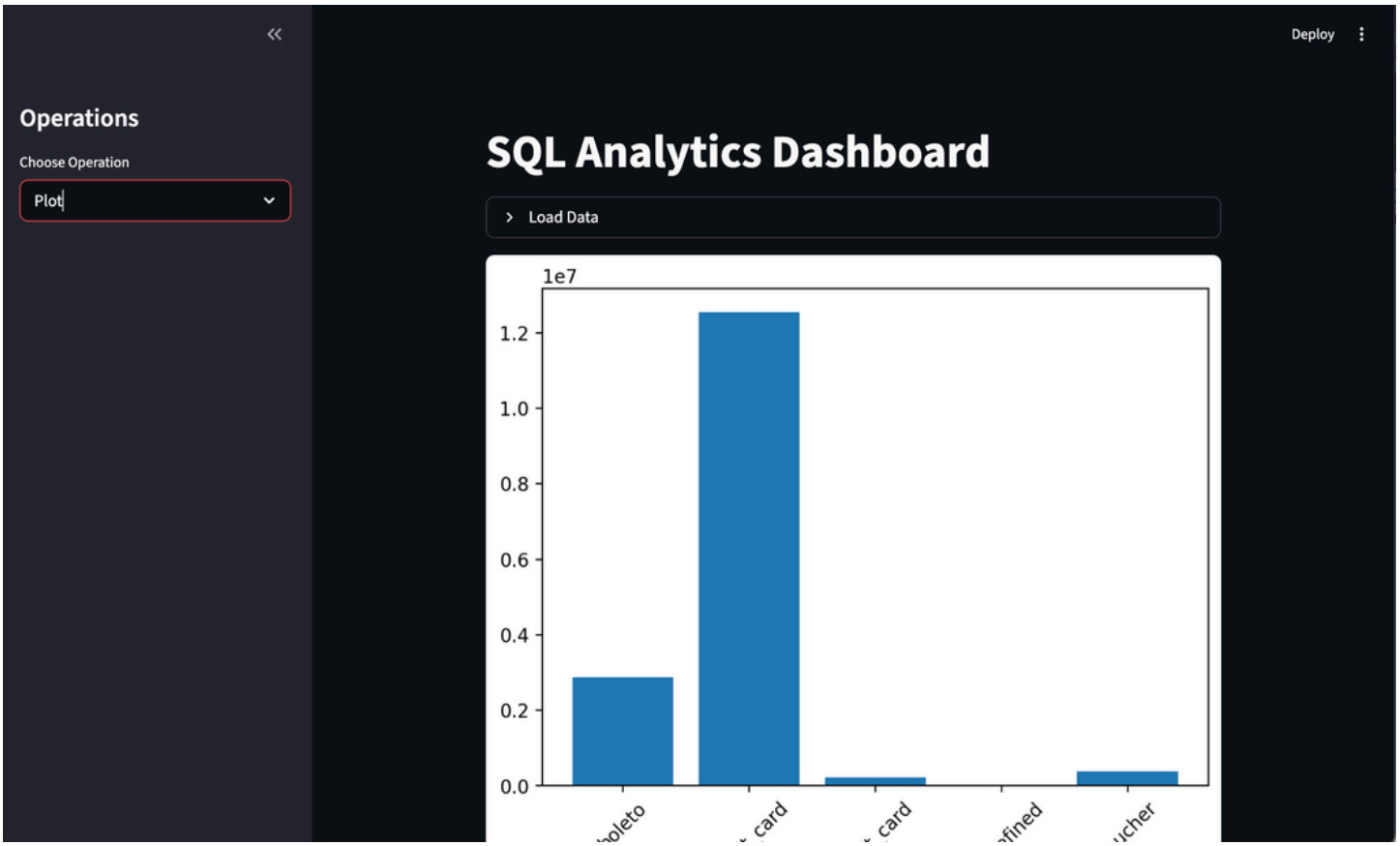
Snapshots attatched regarding project:

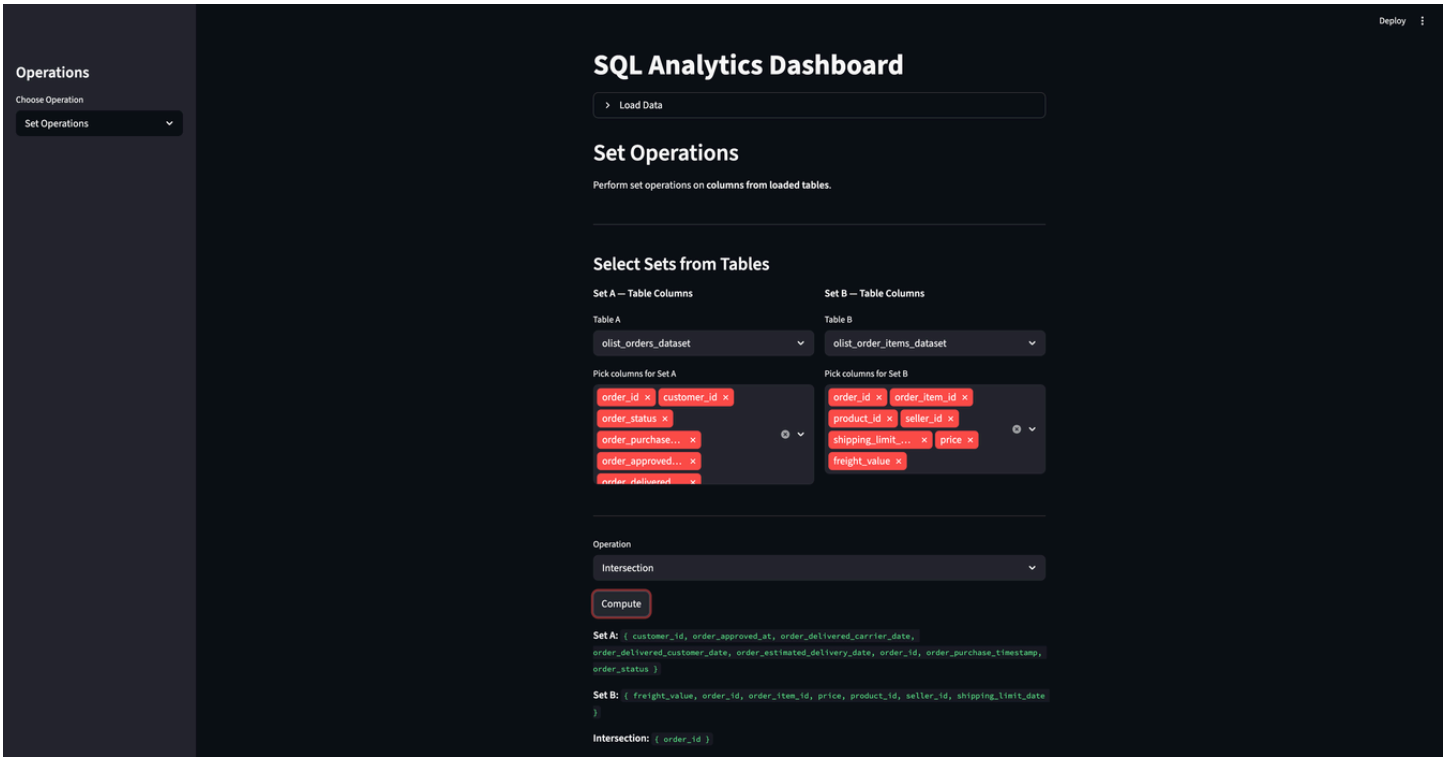

localhost:8501
Finish update
Deploy
SQL Analytics Dashboard
Load Data
Enter folder path with CSV files
Load from Folder
Load some CSV data above to get started.

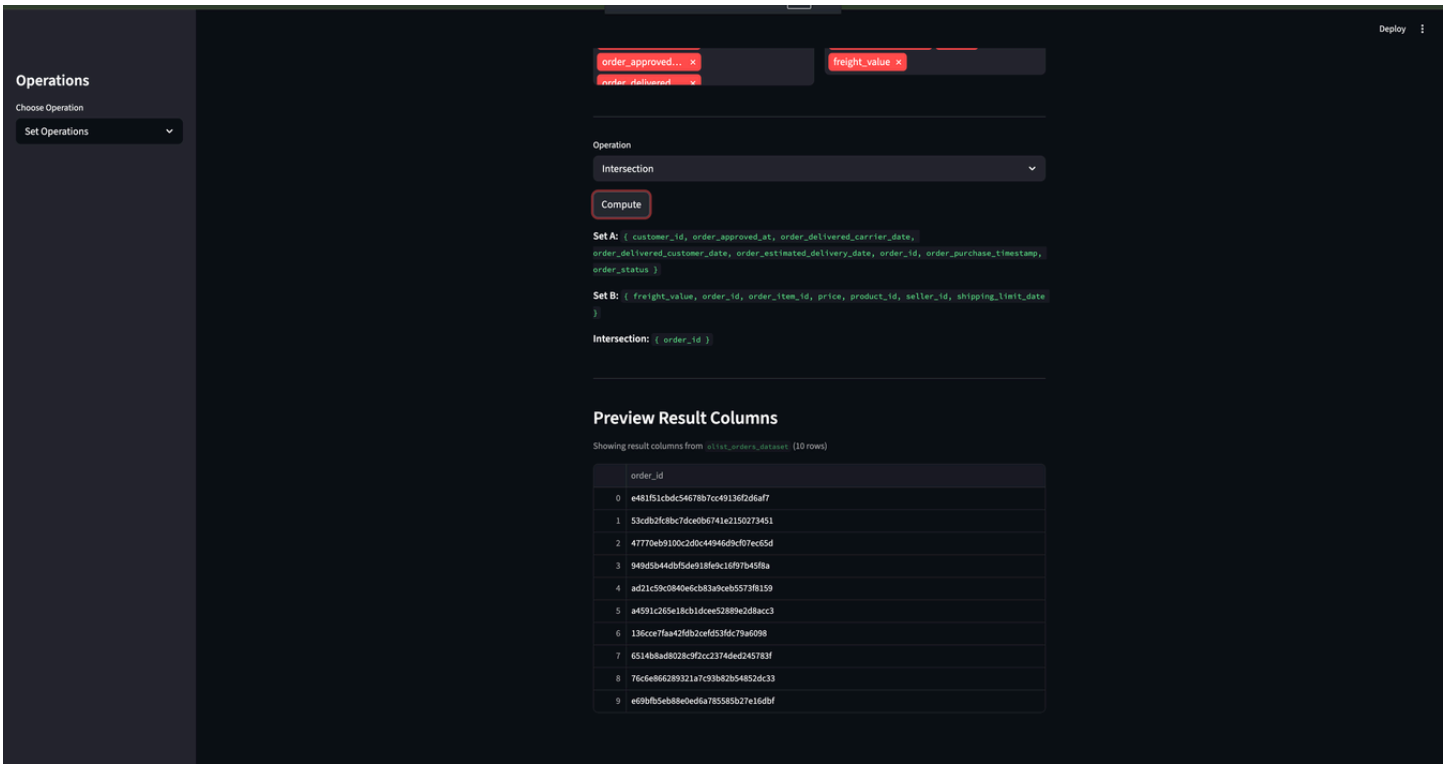

Deploy
Operations
Choose Operation
Display Records
SQL Analytics Dashboard
Load Data
Table
olist_sellers_dataset
Rows
5
Run
seller_id
seller_zip_code_prefix
seller_city
seller_state
0 3442f8959a84dea7ee197c632cb2df15 13023 campinas SP
1 d1b65fc7debc3361ea86b5f14c68d2e2 13844 mogi guacu SP
2 ce3ad9de960102d0677a81f5d0bb7b2d 20031 rio de janeiro RJ
3 c0f3eea2e14555b6faeea3dd58c1b1c3 4195 sao paulo SP
4 51a04a8a6bdcb23deccc82b0b80742cf 12914 braganca paulista SP

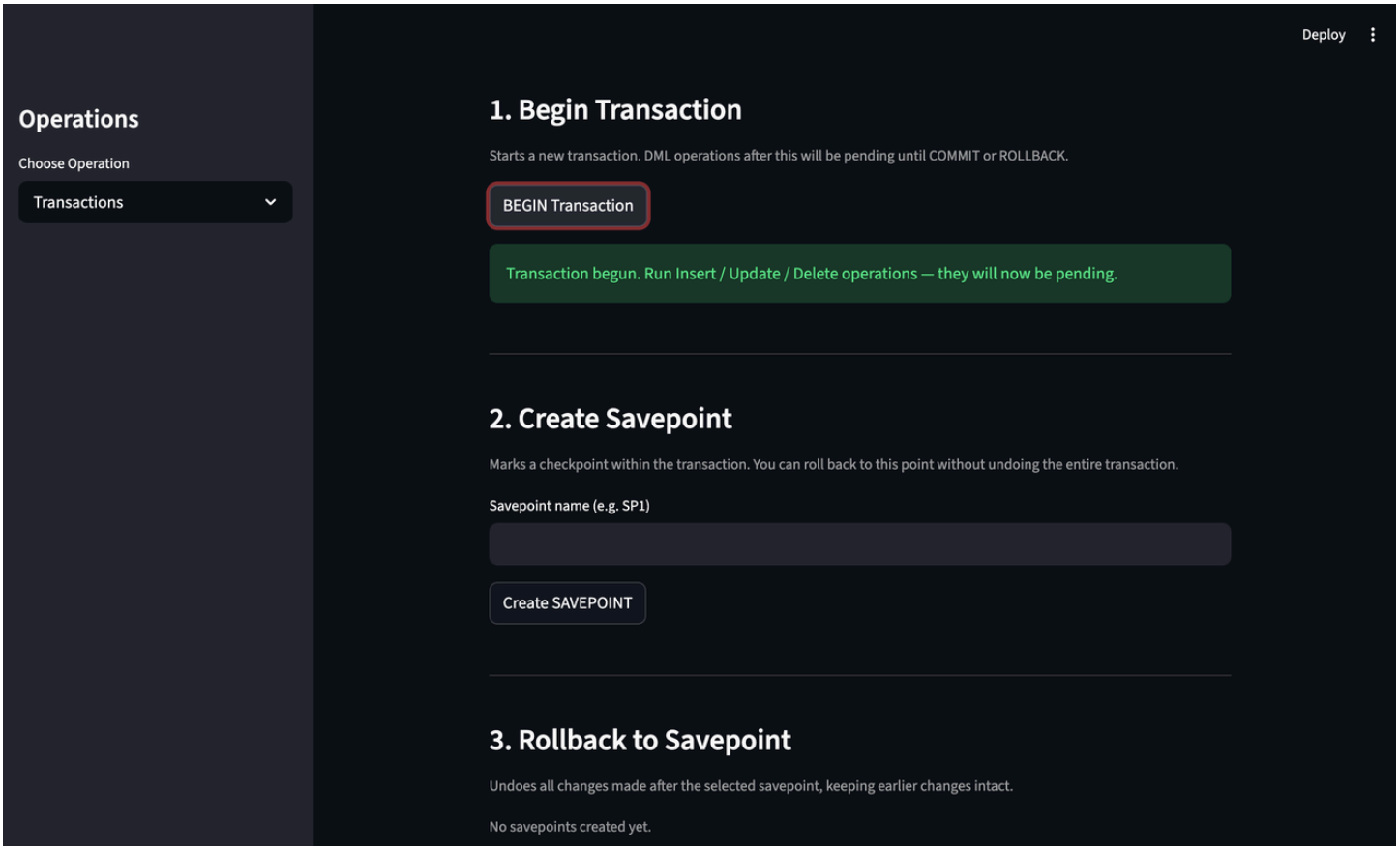

Deploy
Operations
Choose Operation
Filter Records
SQL Analytics Dashboard
Load Data
Table
olist_customers_dataset
Column
customer_city
Value
rio de janeiro
Run
customer_id
customer_unique_id
customer_zip_code_prefix
0 5aa9e4fdd4dfd20959cad2d772509598 2a46fb94aef5cbeeb850418118cee090 20231
1 a7c125a0a07b75146167b7f04a7f8e98 5c2991dbd08bbf3cf410713c4de5a0b5 22750
2 b64ed91eab98972150bdaf77ca921934 3da7750bf3c1dbd724624a60a9f5942b 21310
3 cc32707d2e2f7c92ab449f9b28154809 0d516ca029d6a28d5cfddd80b27a26dc 22440
4 7f2357cfd99c900565e6ffd6d2708d9d 466c3a741a952640c1984a48ebbffbf9 22251
5 21fb2ebc5077f72af04e6ead0763eba5 59150cde6405fb8f31f6dd1882d15754 22290

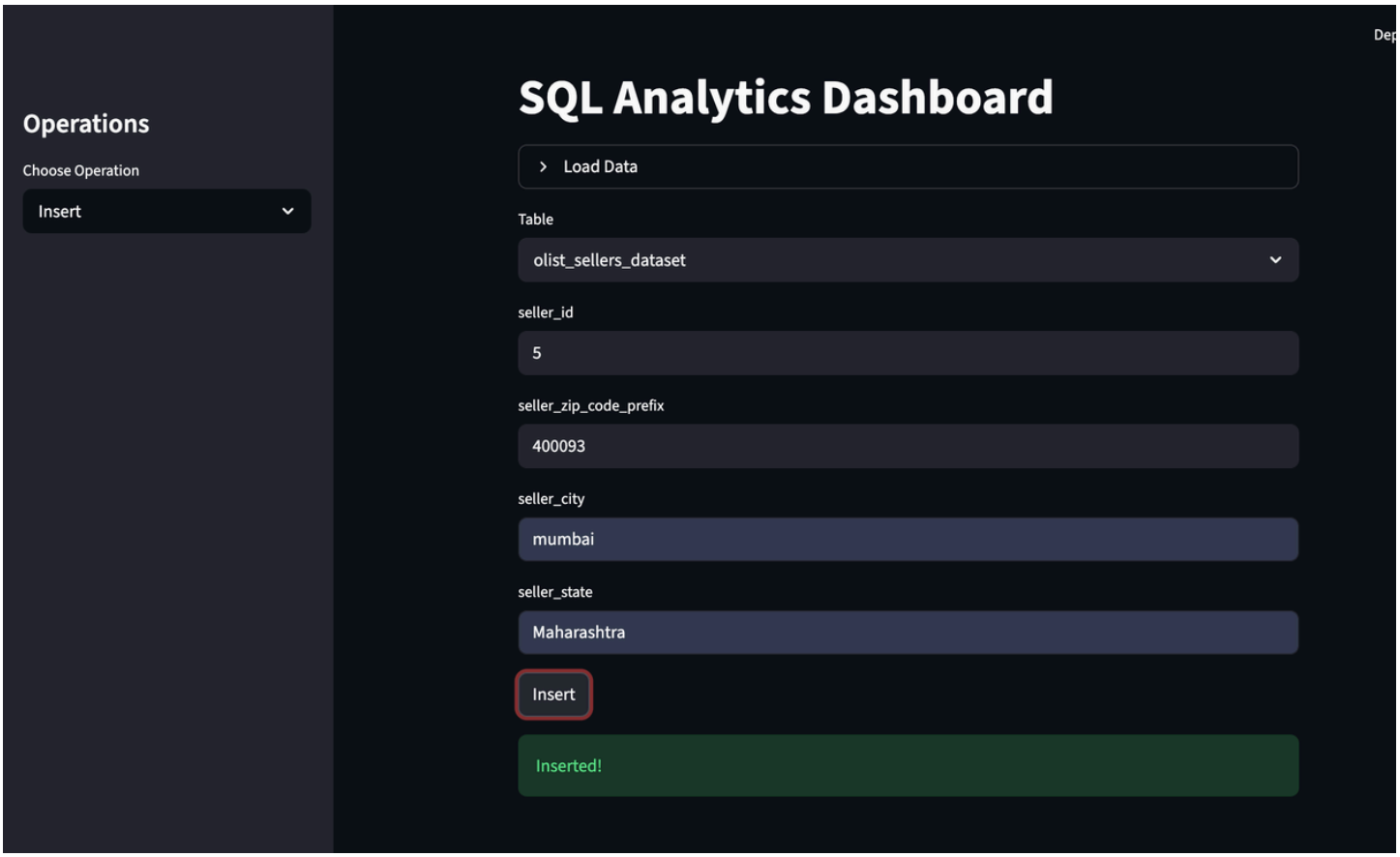

Deploy
Operations
Choose Operation
Group By
SQL Analytics Dashboard
Load Data
Table
olist_order_payments_dataset
Group Column
payment_type
Aggregate Column
payment_value
Function
SUM
Run
payment_type
result
0 boleto 2869361.27
1 credit_card 12542084.19
2 debit_card 217989.79
3 not_defined 0
4 voucher 379436.87

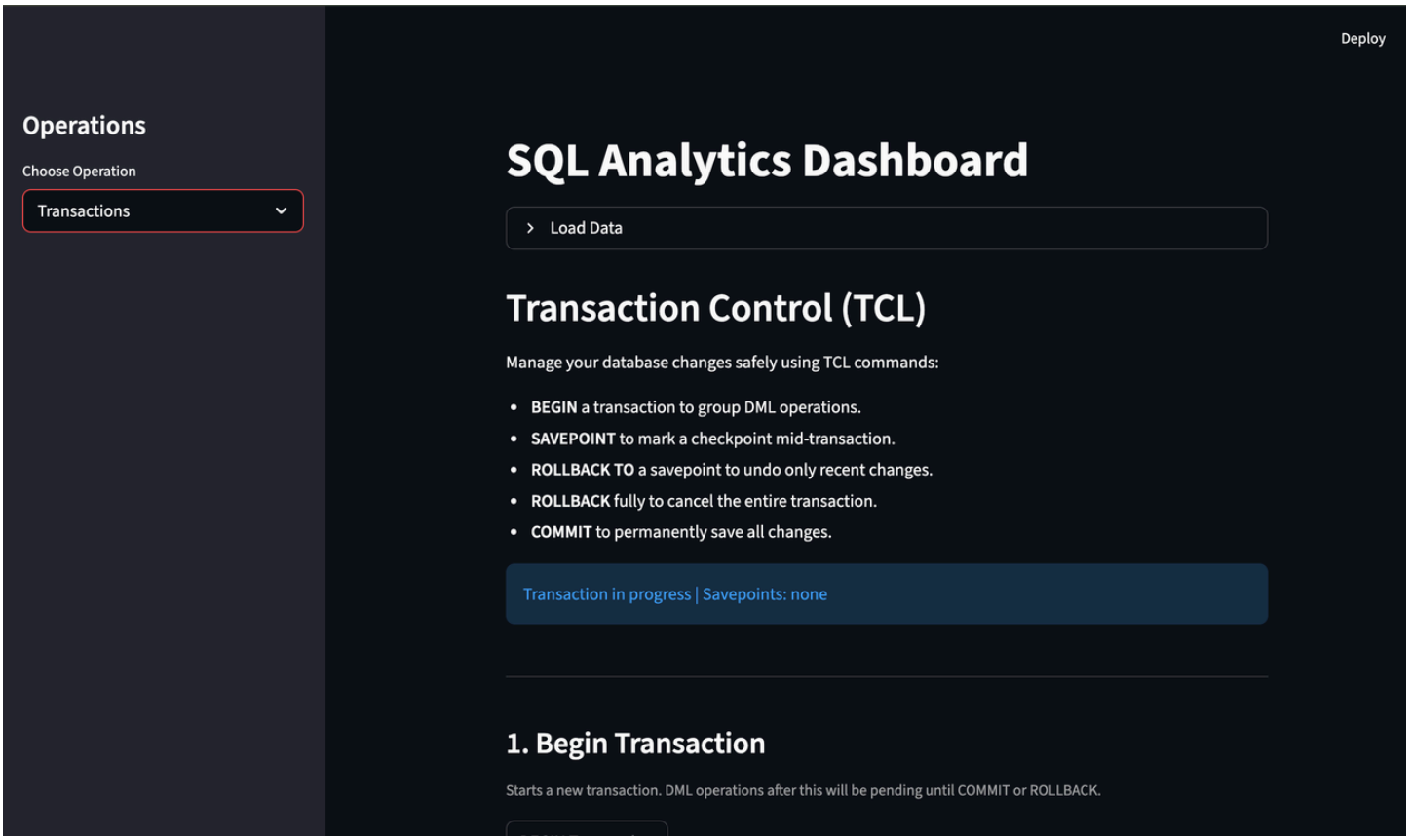

Deploy
Operations
Choose Operation
Plot
SQL Analytics Dashboard
Load Data
1e7
1.2
1.0
0.8
0.6
0.4
0.2
0.0

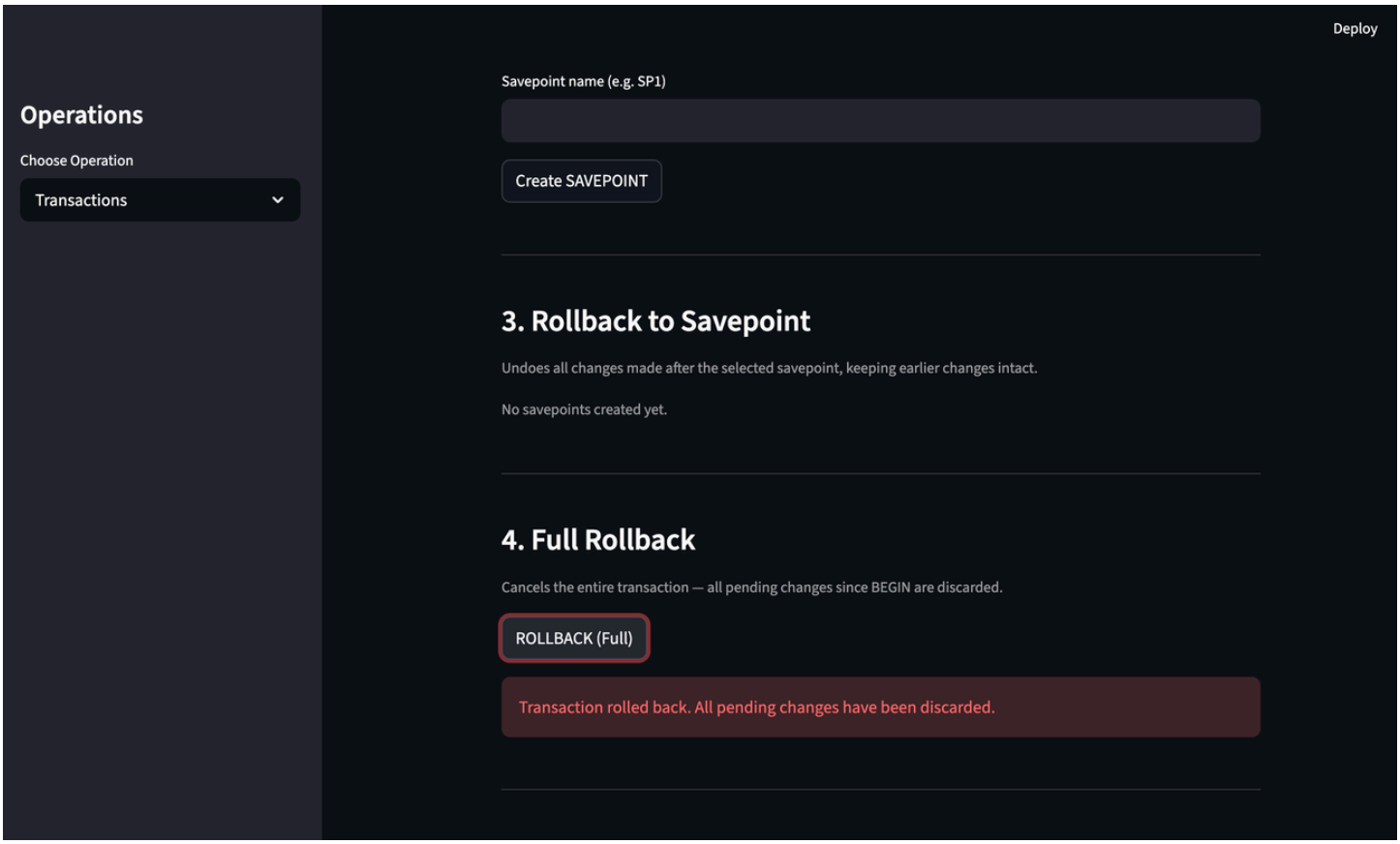

Deploy
Operations
Choose Operation
Set Operations
SQL Analytics Dashboard
Load Data
Set Operations
Perform set operations on columns from loaded tables.
Select Sets from Tables
Set A — Table Columns
Set B — Table Columns
Table A
olist_orders_dataset
Table B
olist_order_items_dataset
Pick columns for Set A
order_id
customer_id
order_status
order_purchase...
order_approved...
Pick columns for Set B
order_id
order_item_id
product_id
seller_id
shipping_limit_...
price
freight_value
Operation
Intersection
Compute
Set A: { customer_id, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date, order_id, order_purchase_timestamp, order_status }
Set B: { freight_value, order_id, order_item_id, price, product_id, seller_id, shipping_limit_date }
Intersection: { order_id }


Deploy
Operations
Choose Operation
Set Operations
order_approved... ×
freight_value ×
Operation
Intersection
Compute
Set A: { customer_id, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date, order_id, order_purchase_timestamp, order_status }
Set B: { freight_value, order_id, order_item_id, price, product_id, seller_id, shipping_limit_date }
Intersection: { order_id }
Preview Result Columns
Showing result columns from olist_orders_dataset (10 rows)
order_id
0 e481f51cbdc54678b7cc49136f2d6af7
1 53cdb2fc8bc7dce0b6741e2150273451
2 47770eb9100c2d0c44946d9cf07ec65d
3 949d5b44dbf5de918fe9c16f97b45f8a
4 ad21c59c0840e6cb83a9ceb5573f8159
5 a4591c265e18cb1dcee52889e2d8acc3
6 136cce7faa42fdb2cefd53fdc79a6098
7 6514b8ad8028c9f2cc2374ded245783f
8 76c6e866289321a7c93b82b54852dc33
9 e69bfb5eb88e0ed6a785585b27e16dbf


Deploy
Operations
Choose Operation
Transactions
1. Begin Transaction
Starts a new transaction. DML operations after this will be pending until COMMIT or ROLLBACK.
BEGIN Transaction
Transaction begun. Run Insert / Update / Delete operations — they will now be pending.
2. Create Savepoint
Marks a checkpoint within the transaction. You can roll back to this point without undoing the entire transaction.
Savepoint name (e.g. SP1)
Create SAVEPOINT
3. Rollback to Savepoint
Undoes all changes made after the selected savepoint, keeping earlier changes intact.
No savepoints created yet.


Operations
Choose Operation
Insert
SQL Analytics Dashboard
Load Data
Table
olist_sellers_dataset
seller_id
5
seller_zip_code_prefix
400093
seller_city
mumbai
seller_state
Maharashtra
Insert
Inserted!


Deploy
Operations
Choose Operation
Transactions
SQL Analytics Dashboard
Load Data
Transaction Control (TCL)
Manage your database changes safely using TCL commands:
BEGIN a transaction to group DML operations.
SAVEPOINT to mark a checkpoint mid-transaction.
ROLLBACK TO a savepoint to undo only recent changes.
ROLLBACK fully to cancel the entire transaction.
COMMIT to permanently save all changes.
Transaction in progress | Savepoints: none
1. Begin Transaction
Starts a new transaction. DML operations after this will be pending until COMMIT or ROLLBACK.



VII. Self -Learning beyond classroom

This project entailed more than just acquiring a few practical skills on my own by going beyond the theoretical concepts that had been discussed in class.

The course did not discuss SQLite3 at length, and thus knowing how to programmatically interface with SQLite using the Python sqlite3 module meant having to read the documentation. Among the most important things I learned throughout the journey were how to manage an in-memory database connection during a Streamlit session, how to play with isolation level to allow manual control of transactions and how to execute raw SQL with parameterized queries to prevent injection problems.

Streamlit was a self-educated person. Understanding the functionality of st.session_state was especially useful to this project because the database connection, transaction status, savepoints and logs should be able to survive user interactions without being set back to default. The most important sources of information were the official Streamlit documentation and community forum.

Pandas was not new to me by other coursework but needed supplementary reading to learn how DataFrames can be translated to database tables and how to use type inference properly in loading CSVs because it was being used as an intermediary between SQLite and the UI, i.e. read_sql and to_sql.

The plotting module was plotted using Matplotlib. The Matplotlib site was consulted to find documentation and examples on how to correctly render the bar chart using st.pyplot in Streamlit.

Locating a suitable data was a challenge unto itself. The need was a relational in nature dataset (i.e. more than one table, with significant foreign key relationships among them) such that joins, aggregations, and

normalization would be non-trivial and realistic. The main search engine was Kaggle. The datasets were assessed in the following ways: the quantity of tables, the degree to which the schema has already been normalized, and the quality of data such that it could be loaded without much preprocessing. The final dataset that was selected was the Olist Brazilian E-Commerce that provided nine interrelated tables containing a type of data that was diverse enough to showcase the entire set of operations incorporated into the dashboard.

:VIII. Learning from the Project

Learning and Takeaways

Deepening Course Concepts

The creation of a functional application based on the theory of DBMS obliged me to some extent of knowledge that is usually not needed in examinations and other assignments. Ideas learned in an abstract sense needed to be applied accurately and correctly, which revealed weaknesses and solidified knowledge in a manner that is not available in passive learning.

The most obvious example of this was normalization. Although the 2NF and 3NF rules are easy to memorize, using a tool to automatically identify any violations, derive candidate keys based on attribute closure, decompose a schema, and finally ensure that dependencies are maintained, it is still necessary to understand every step of the process at a mechanical level. The logic or not was either black or white, there was no place to indulge in a hazy feeling.

There was also a shift of transaction control to the concrete. Only when we needed to maintain an SQLite connection open between two or more user interactions, record each operation, and either commit or roll it back did we realize what it meant to say that a change is pending. The distinction between a savepoint rollback and a full rollback was instantly intuitive as soon as it had to be written down as code.

The abstract concepts of relational algebra such as set operations, which may seem unrealistic in a classroom context when taught through textbook and lecture, were brought to life through the mapping onto the real table columns and implemented on the real data.

Already known were joins and aggregations but implementing them using a dynamic interface where the table, columns and functions are all selected dynamically demanded a better understanding of how SQL queries are built and the flow of results through the system.

Some of the skills and tools applied in this project were acquired all by themselves.

The course did not cover SQLite3 in great depth and therefore, knowing how to use it programmatically by utilizing the Python sqllite3 module entailed reading through the official documentation. Among the most important lessons was how to manage an in-memory connection to a database throughout a Streamlit session, how to deal with isolation level to allow manual control of transactions, and how to run a raw SQL query with parameterized queries.

Streamlit was a self-taught person. Of interest was learning how st.session state works because all the database connection, state of transaction, savepoints, and logs had to be preserved across user interactions without getting reset. The main resources were the official Streamlit Docs and community forum.

Pandas was not new to other courses but applying it as an SQLite-UI bridge, namely read_sql and to_sql, necessitated some more reading to figure out how DataFrames are represented in database tables and how type inference operates when loading CSVs into a running database.

The plotting module was plotted with Matplotlib. The Matplotlib site was consulted to find documentation and examples that would allow bar chart rendering to work with Streamlit using st.pyplot.

It was difficult to locate a suitable dataset, in the first place. This was the need to have a dataset that was relational in nature (several tables with meaningful foreign key relationships) such that joins, aggregations, and normalization would be not trivial and realistic. The main search site was Kaggle. Data was assessed by the number of tables, the schema was already normalized and the data was clean enough to be loaded without any major preprocessing. The Olist Brazilian E-Commerce dataset was eventually selected as it presented nine interlinked tables that had diverse types of data and hence was applicable in illustrating the entire set of operations embedded into the dashboard.

Overall

The project was a good experience in learning the distinction between knowing a concept and being able to put it to use. Being forced to make all of DBMS theory functional, interactive, and being forced to learn how to do it on my own made the topic much more concrete, and the practical aspect of the learning that this forced me to pursue alone also gave a practical side to the course that would have been lacking without it.

IX. Challenges Faced

Handling the Database Connection between Sessions.

The most irritating and difficult challenge was maintaining the SQLite connection alive between user interactions in Streamlit. Streamlit will rerun all of the code each time a user takes an action, and since a naive initialisation of a connection will destroy it and create a new one, the in-memory database will be wiped out every button press. The fact that st.session_state might be used to continue the connection object through reruns was not clear at first and had to be discovered by sifting through the documentation of Streamlit and community forums before reaching a working solution.

Use of Manual Transaction Control.

SQLite has default settings of operating in autocommit, and to override this to use manual BEGIN, COMMIT, and ROLLBACK mode, one needs to understand the isolation level setting, not well covered in introductory material. isolation level = None placing the control of hands full transactions at the setting of the developer, but this further implied that all DML operations would have to be handled with care - either a transaction was in progress, and the changes would have to remain pending, or the changes would have to be committed immediately. This logic required a lot of debugging to get consistent with the Insert, Update and Delete operations and to be able to maintain an accurate transaction log.

Savepoint Rollback State Manager.

Savepoints added a new dimension of complication to the transaction system. To roll back to a savepoint, not only was the state of the database to be restored, but the application internal savepoint list also had to be pruned appropriately - any savepoints made since the one being rolled back to are no longer valid. It was a bit of a struggle to get this synchronization correct between the internal state of SQLite and the st.session state. savepoints and early releases of the feature used to leave the UI in inconsistent states with invalid savepoints still being available to select following a rollback.

Creating the Normalization Engine Scaffold.

By far the most algorithmically challenging part of the project was the normalization calculator. No libraries to do 2NF and 3NF analysis exist - all of the logic was to be done in first-order. Computing attribute closure, locating all candidate keys by trying all subsets of attributes, locating partial and transitive dependencies, decomposing the schema, followed by checking dependency preservation among the resulting relations all had to be written and debugged separately. The first incarnations of the candidate key finder were too slow on large sets of attributes due to the brute-force subset enumeration, and the decomposition reasoning generated redundant or erroneous relations in edge cases that would require several iterations to resolve.

Extracting Functional Dependencies.

Normalization calculator takes in the functional dependencies as free-form text. Coming to terms with this input consistently, such as dealing with differing whitespace, comma separations, multi-attributes left and right hand sides, and gracefully handling bad entries was more complex than anticipated. Determining that all the attributes mentioned in an FD do indeed exist in the relation schema, and providing clear error messages without bringing down the app, meant that their input handling had to undergo several iterations.

Storing Query Results to the Plot Module.

The Plot module relies on the output of a previous Group By or Join operation, but since Streamlit restarts the script each time you interact with it, the DataFrame created by one operation is not necessarily available to other operations. This was solved by storing the last query result in st.session_state but it was necessary to make decisions when to update it and to make sure that the plot always showed the most up to date relevant query and not stale data.

Real Table Data Set Operations.

The first module that seemed easy to use was the Set Operations module - Python sets already include union, intersection, and difference. Making the results meaningful and previewable in the context of actual database tables was the challenge. The result of an operation had columns

that were present in many tables, so there was no individual table to query to preview the result, which needed to be identified and reported to the user in a conspicuous way and not quietly fail or give the wrong answer. The edge cases (empty sets, overlapping sets of columns, and results that overlapped tables) involved a series of trials against the real dataset.

Utilizing CSV Data Reliably.

Importing arbitrary CSV files into SQLite using Pandas appears easy at first but the reality was that there was inconsistency in the naming of columns, column data types, and files with issues with their encoding. To make sure that all files loaded into an accurately typed SQLite table, and column names did not include any character that would ruin SQL queries, it was necessary to add defensive handling around the load process, which had not been originally expected.

X. Conclusion

The project aimed to illustrate an applicable knowledge of the essential ideas taught in the Database Management Systems course, and in the process became one of the more challenging and fulfilling projects in the semester. Instead of education about theory per se, the aim was to create something workable, an application where each concept needed to be put to the test, fully and properly and in a manner that could be interacted with and tested in real time. That test was very much a higher test than any written test, and it needed a level of comprehension that could not have been achieved through passive study.

The scope of what the application covers directly relates to the scope of the course. Primarily, basic data manipulation, aggregation, joins, set operations, transaction control, and normalization are not periphery issues, they form the basis of relational database theory and each of them is displayed in the dashboard as a working, useful module. Their application in a series throughout the project provided a cumulative effect of understanding with each idea strengthening another. Without a solid understanding of functional dependencies, attribute closure and the candidate key theory, the normalization calculator, e.g., would not have been possible to build. Without knowledge of the semantics of atomicity and the functionality of savepoints in partial recovery, the transaction module would not have been possible. The application, as a whole, is organized in the same manner that the subject itself is organized - one layer on top of another.

The decision to employ the Olist Brazilian E-Commerce dataset was also carefully thought. The schema was complicated enough to ensure that each operation in the dashboard was meaningful by a dataset of nine interrelated tables that included customer, orders, product, seller, payment, review, and geolocation data. Orders to customer joins, payment values aggregation, product and seller data normalization analysis were not examples - they were operations on a schema that models the layout of a real commercial database. This based the project on some degree of practical relevance that a synthetic or minimal dataset would not have provided.

Along with the technical implementation, a lot of independent learning was needed in the course of this project. The course curriculum did not include tools and libraries that would otherwise have been covered, such as Streamlit, the programmatic interface to SQLite3, and the bridging functionality of Pandas between DataFrames and live database tables, so they would have to be learned adequately to be used in a production-grade environment. The main sources of the acquisition of that understanding were documentation, community

resources and progressive debugging. Being able to determine what had to be learned, finding the appropriate resources and implementing new knowledge under the limitations of a working project is a skill in itself and one that was significantly built by this project.

The issues it faced on the way, such as dealing with persistent state in a script-based execution model, providing manual transaction control at the connection level, constructing a normalization engine out of first principles, and ensuring uniform behavior across all modules were not accidental. They formed the content of the learning. Every challenge necessitated a re-examination of basic principles, prudent thought on what the right action ought to be, and then constructing up to it. The outcome is an application that is not just simulating the DBMS operations, but actually executing them, on a real database, with real data.

Finally, QueryLab proves that the ideas studied in this course are not theoretical, or scholarly, in nature, but rather the principles that all serious data-driven systems are built on. The distinction between knowing what a transaction is and knowing why transactions are designed the way they are is understanding them on the level needed to implement them. The distinction between being able to define third normal form and being able to test whether it is violated, in an algorithmic manner. It is that difference, between surface familiarity and real understanding, which this project, more than any other, has led to the establishment of.